

Build a Matching Engine on Windows

Standalone teaching and progress workbook

Native Windows | MSYS2 UCRT64 g++ | C++23 | CMake | Ninja | GDB

Build a price-time-priority matching engine you can explain, test, measure, and extend. Complete L01-L12 for the first release. Then choose performance, protocol/networking, concurrency, or recovery extensions. Existing correct work can satisfy a lesson; you do not need to rewrite it.

Read, predict, implement, verify

Each lesson begins with teaching pages that explain the concepts, define the needed interfaces or formats, and include a worked exercise with its answer. Its practice page follows with implementation steps, checks, and fillable evidence fields. L02 also includes complete starter files and commands. Read each lesson in that order.

The necessary teaching is inside this PDF. Numbered source links identify factual references and optional further detail. You do not need to open them to complete the stated lessons. Installers and the initial GoogleTest download still need internet access. X01 includes synthetic ITCH fixtures so a purchased or downloaded market-data file is not required.

Use the fillable tracker

1. Open the PDF in a form-capable reader and save a working copy. Use the status dropdowns in the core and extension trackers: New, Active, Blocked, Done; extensions also offer Deferred.
2. Keep one lesson Active. Before stopping, record a precise next action and any blocker in the session log.
3. On each practice page, mark understanding and verification separately. Record a test, trace, benchmark file, or commit plus actual minutes.
4. Save after each session. Counts are manual. If the preview does not activate fields, download and use a form-capable PDF reader.

Know the scope of this edition

This edition uses g++ throughout. The diagnostic configurations are Debug, Checked libstdc++ containers, and Release tests. Checked adds container and iterator validity checks; L02 explains its coverage. Windows commands and network/profiler behavior still need to be run on your own Windows installation; the workbook does not claim those steps have already passed on your machine.

The contract pages remain the authority for the core. Extension teaching pages specify additional bounded-capacity and protocol policies where needed. Those additions do not change the first release's behavior unless you select that branch.

Navigate

Use the PDF bookmarks or the lesson links in the two trackers. They open the beginning of each lesson's teaching. The end-of-lesson practice page retains the progress fields. Begin with L01, or verify its checks against your existing setup and proceed to L02.

Definitions and how to follow the explanations

This workbook builds an order matching program. An order is a request to buy or sell a stated amount. A matching engine applies explicit rules to pair compatible buy and sell orders. Start with these definitions before the contract and lessons. Each lesson then introduces its additional vocabulary before using it in an implementation exercise.

The market model used in this project

Term	Meaning here
Instrument / symbol	The thing being traded, identified by a name such as TEST. The first release has one synthetic instrument.
Side	Buy or Sell, the two possible directions of an order.
Price / tick	Price is an integer number of ticks. A tick is the chosen smallest price step; no dollar conversion is assumed.
Quantity / remaining	Quantity is a number of units; remaining is the part of an order not yet executed or removed.
Limit order	An order allowed to buy at its limit or lower, or sell at its limit or higher. Unexecuted quantity may wait.
Market order	An order with no limit-price barrier in this simulator. Unexecuted quantity expires after available opposite orders are exhausted.
Resting order / order book	A resting order waits for an opposite order. The order book is the owned collection of those waiting orders.
Bid / ask / BBO	A bid is a buy order; an ask is a sell order. Best bid is the highest bid; best ask is the lowest ask. BBO is their pair.
Price level / depth	A level groups resting orders at one price on one side. Depth is their summed remaining quantity.
Crossing	A buy limit at least as high as an ask, or a sell limit at most as low as a bid, permits an execution.
Maker / taker	The maker was already resting; the taker is the incoming order seeking a match. Aggressor is another name for taker.
Trade / execution / fill	A trade records matched units at a price. Executing or filling an order removes those matched units from its remaining amount.
Partial fill / residual	A partial fill leaves units unexecuted. The residual is that leftover amount; it either rests or expires under the order rule.
FIFO / priority	First In, First Out means earlier orders at the same price go first. Priority is their ordering position, represented by a counter.
Cancel / amend	Cancel removes remaining units. Amend requests a new price or new remaining quantity under the workbook's priority rules.

The program's state and interfaces

State means the values the program currently stores. Mutation changes those values. A command is a typed request to change state. A result reports whether the request succeeded and what it did. A rejection is an ordinary denied request; it must not partially change the book. An error while allocating memory is a different failure class, covered by an explicit instance-failure policy.

An ID, or identifier, is a value used to distinguish an object. An active ID identifies an order still resting. A sequence number counts events in a declared order; it is not an order ID or a clock reading. A session is one identified run or connection whose counters have a shared context. A timestamp is a clock reading associated with an event.

An interface specifies how one component can call another: parameter types, return types, and promised behavior. API means Application Programming Interface, the set of such callable operations. A component is a part with one responsibility, such as the book or parser. An adapter converts an outside representation into the core's types or converts results back out. A CLI is a command-line interface that accepts arguments or text and prints results.

Ownership means responsibility for keeping an object alive and eventually destroying it. Lifetime is the interval during which that object exists. A handle is a value that refers to an owned object, such as an ID or iterator; it does not necessarily keep the object alive. A locator stores where a live order resides. A dangling handle refers to an object whose lifetime ended. A snapshot is an owned copy of state at a stated boundary. Canonical means encoded or ordered in one declared, repeatable way.

An invariant is a property that every legal state must satisfy. A precondition must hold before an operation; a postcondition must hold afterward. Preflight checks all required bounded resources before mutation. Capacity is the maximum number of items or bytes a component can retain. An aggregate is a sum over individual items. Overflow means a mathematical result exceeds the type's representable range; unsigned arithmetic wraps unless you prevent it.

C++ notation used in the workbook

Notation	Meaning before you use it
<code>std::</code>	Select a name from the C++ standard-library namespace. A namespace groups names and avoids collisions.
<code>#include <...></code>	Make declarations from the named library header available to this source file. A header is a file intended to be included.
<code>#pragma once</code>	Ask the compiler to include that project header once within a translation unit, avoiding duplicate definitions.
<code>using Name = Type</code>	Create an alias, a second name for the same type; it does not create a separately checked type.
<code>int64_t / uint64_t</code>	Exactly 64-bit signed/unsigned integer types from <code><stdint></code> . <code>uint8_t/u16</code> notation elsewhere means an unsigned integer of that width.
<code>struct / class</code>	Define a type that groups data and functions. A struct starts with public members; a class starts with private members.
<code>enum class</code>	Define a named set of values, such as <code>Side::Buy</code> and <code>Side::Sell</code> , distinct from ordinary integers.
<code>const / auto</code>	<code>const</code> prevents changes through that declaration; <code>auto</code> asks the compiler to deduce the type from its initializer.
<code>T& / const T&</code>	A reference names an existing object; <code>const T&</code> gives read-only access through that reference. It is not an owned copy.
<code>T* / &object / *pointer</code>	A pointer stores an address; <code>&</code> obtains an address; <code>*</code> accesses the pointed-to object when it is valid.
<code>object.member / pointer->member</code>	Select a member directly or through a pointer.
<code>constructor / destructor</code>	A constructor initializes an object; its destructor, written <code>~Type()</code> , releases resources when its lifetime ends.
<code>= delete</code>	Forbid an operation such as copying a resource owner. This makes an attempted use a compilation error.
<code>template<...></code>	Parameterize a type/function by another type or a compile-time value. <code>Ring<4></code> uses an array size fixed at compilation.
<code>lambda [captures](parameters){body}</code>	A local callable function. <code>[&]</code> refers to surrounding objects; they must outlive the lambda's use.
<code>static_assert / assert</code>	<code>static_assert</code> checks during compilation. <code>assert</code> checks at runtime unless <code>NDEBUG</code> disables it.
<code>throw / exception</code>	Signal an exceptional failure that transfers control to a matching handler. Destructors release owned local resources during normal unwinding.

The sample namespace `lob` abbreviates limit order book. Function names such as `process`, `submit`, and `snapshot` in code labeled 'pattern' or 'sketch' describe roles to map to your interface; they are not hidden supplied functions. Complete miniatures identify their headers and can be compiled as exercises. Pseudocode is language-independent logic, not a C++ source file.

Numeric notation and code operators

A byte contains eight bits. Decimal is base ten; a prefix `0x` denotes base sixteen (hexadecimal), using digits 0-9 and A-F. Each pair of hexadecimal digits encodes one byte: `0xFF` is 255. `UINT64_MAX` is the largest unsigned 64-bit value. A literal suffix `u` requests an unsigned integer. A zero-based index starts at zero, so the last index of `n` elements is `n-1`.

In C++, `<<` and `>>` shift integer bits; `&` combines bits with AND; `|` combines them with OR; `^` is exclusive OR. These operators are different from stream insertion `<<` when the left operand is an output stream. `%` is remainder. `&&` and `||` are short-circuit Boolean AND/OR, so a failed first condition can prevent an invalid later access. `++` increments; `--` decrements. `std::min` selects the smaller value. `std::to_integer` converts a byte to an integer. A `static_cast` explicitly requests a conversion; range-check first when information could be lost.

Containers and helper types before the first code sample

Type or operation	Meaning
vector<T> / array<T,N>	A vector owns a resizable contiguous sequence; array owns exactly N elements. Contiguous means adjacent in memory.
deque<T> / list<T>	A deque supports end operations with segmented storage. A list owns linked nodes, with links connecting consecutive elements.
map<K,V> / unordered_map<K,V>	Both associate unique keys with values. map traverses keys in comparator order; unordered_map uses hashing and has no sorted iteration order.
Comparator / hash / bucket	A comparator decides key ordering. A hash maps a key to a lookup value; a hash-table bucket groups entries selected from that hash.
string_view	A non-owning view of characters; it does not copy the string or extend its lifetime.
optional<T> / nullopt	optional either contains one T or has no value; nullopt denotes absence. Check before dereferencing.
variant<A,B> / span<const byte>	A variant holds one of the listed types. A span is a non-owning view of a bounded sequence; const byte makes the viewed bytes read-only.
Iterator / begin / end	An iterator identifies a container position. begin starts traversal; end is the past-the-end marker and must not be dereferenced.
size / empty / front	size counts elements, empty checks for zero elements, front accesses the first element and requires nonempty storage.
push_back / emplace	push_back appends a value; emplace constructs an entry from arguments.
erase / clear / reserve	erase removes specified entries; clear removes all elements; reserve prepares capacity without creating those elements.
find / at / contains	find locates a key or returns end; at accesses an existing key/index with checking; contains tests key membership.
std::byte / size_t	byte represents raw byte storage; size_t is an unsigned type used for object sizes and container indices.

Build, test, and performance language

A compiler translates source into object code. A linker combines object code and libraries into an executable. g++ is the GNU C++ compiler driver; GCC is its compiler collection. C++23 names the 2023 C++ language standard. MSYS2 supplies development tools on Windows; UCRT64 selects its 64-bit native tool family using Microsoft's Universal C Runtime, the base C runtime library. A DLL is a Windows dynamic-link library loaded at runtime.

CMake describes targets, which are named build products and their dependencies. Ninja executes generated build instructions. GDB is the GNU debugger for inspecting program execution. VS Code is the editor. PATH is the list of directories searched for executable commands. A build cache stores configuration choices; an ABI, or Application Binary Interface, defines how compiled components represent data and call each other. Mixing incompatible container representations breaks that agreement.

A test fixture supplies a known starting state or input. A regression test preserves a previously failing case. A reference model is a separate simple implementation used as an oracle, meaning an expected-behavior source. Differential testing compares both implementations. A seed initializes a random generator; replay runs saved concrete commands again. A sanitizer instruments certain operations to diagnose particular errors. Checked containers instead add library-level validity checks and have narrower coverage.

A benchmark measures a declared workload. Throughput counts completed operations per time unit. Service time is the time inside a chosen processing boundary. Latency is elapsed time between two specified events and may include waiting. A percentile is a rank in observed measurements; L12 derives its calculation. A profile attributes observed cost to parts of execution. CPU means central processing unit; an OS, or operating system, manages processes, threads, memory, and devices. Cache locality describes how closely accesses reuse nearby or recent memory. No speed claim follows from locality alone.

The notation $O(f(n))$ describes how a cost is bounded as input size grows, ignoring constant factors. Expected complexity averages over a probabilistic assumption; amortized complexity averages cost across a sequence of operations. Neither means every individual call has that cost. In this workbook N counts active orders, P occupied price levels, and K visited makers unless a lesson states otherwise.

How explanations establish the next step

Each worked example states the initial state, incoming operation, intermediate effects, and final expected state. When you see a minimum, bound, or early exit, identify the precondition that makes it valid. For example, $q = \min(a, b)$ with nonnegative a, b proves both $a - q$ and $b - q$ remain nonnegative. The sweep proof builds on that one-maker fact; the optimization lessons preserve the same contract rather than changing its meaning.

The contract uses these definitions to freeze behavior. Networking and concurrency introduce their own terms at the start of their lessons, and their new byte formats or handoff rules are defined before implementation. When a branch refers to an earlier lesson, its prerequisite names the needed work. The included answer checks your reasoning; your completion evidence demonstrates the implementation.

Core progress tracker

Complete L01 through L12 for the first release. Reuse existing code when it passes the checks. Update the count by hand; there is no automatic scoring.

Core lessons done out of 12 / 12

Current lesson and next action

ID	Topic	Status	Done date
L01	Set up the Windows toolchain	Done	
L02	Build a library and one test	Done	
L03	Represent the passive order book	Done	
L04	Match one resting order	Done	
L05	Sweep price levels and handle markets	Done	
L06	Cancel by ID and formalize invariants	New	
L07	Use stable locations for cancellation	New	
L08	Implement replacement semantics	New	
L09	Build an independent reference and generator	New	
L10	Test failure paths on Windows	New	
L11	Replay commands and preserve evidence	New	
L12	Establish a reproducible benchmark	New	

Done means both lesson checks are complete and evidence is recorded. Blocked means a named issue prevents the next implementation step. Record the issue in the session log.

First release date

Extension progress tracker

Choose a branch after the core. P lessons are performance experiments. E and N lessons build the networked simulator. C lessons introduce concurrency. X lessons integrate or extend it. Deferred extensions do not reduce core completion.

ID	Topic	Status	Done date
P01	Measure allocation and try a pool	New	
P02	Build indexed order storage	New	
P03	Compare price layouts and profile on Windows	New	
E01	Specify a portable binary format	New	
E02	Decode bytes and reject malformed records	New	
E03	Separate responses from market data	New	
N01	Build a Winsock echo exercise	New	
N02	Connect a framed client to the engine	New	
N03	Serve multiple clients with readiness polling	New	
N04	Publish a sequenced UDP feed	New	
N05	Recover a feed from a consistent snapshot	New	
N06	Add multicast and measure batching	New	
C01	Build a bounded queue with a mutex	New	
C02	Move work between owning threads	New	
C03	Implement a single producer ring	New	
C04	Explain publication with acquire and release	New	
C05	Partition symbols across matching owners	New	
X01	Reconstruct a market data book from ITCH	New	
X02	Restart from a journal and snapshot	New	
X03	Demonstrate the integrated Windows simulator	New	

Suggested order: P01 to P03 for performance; E01 to E03 then N01 to N06 for networking; C01 to C04 for queues, then C05 for sharding. X01, X02, and X03 list their own prerequisites.

Matching rules for the core

These are the simulator rules you implement and test. They define a small continuous market. Real exchanges may use different order types, priority rules, or replacement semantics. Freeze this contract before optimizing storage.

Decision	Project rule
Instrument	One synthetic symbol, TEST. Orders represent integer units.
Price	Price is int64_t ticks. Valid limit prices are 1 through 1,000,000,000. A tick is an abstract price increment, not an assumed dollar amount.
Quantity	Quantity is uint64_t. Each new or amended remaining quantity is 1 through 1,000,000,000. Parse signed input before conversion; reject negative values and out-of-range text.
IDs	OrderId is uint64_t in 1 through UINT64_MAX. It must be unique among active resting orders. Reuse after fill or cancel is allowed. Command sequence numbers remain separate and unique within a log.
Priority	Best price first, then arrival priority at that price. Buy prices descend; sell prices ascend. A newly resting order joins the tail.
Execution	A buy crosses when its limit is at least the best ask; a sell crosses when its limit is at most the best bid. Trade quantity is the minimum remaining quantity of the pair. Trade price equals the resting price.
Residual	Unfilled limit quantity rests. Unfilled market quantity expires. Market orders never become resting orders.
Cancellation	Cancel removes all remaining quantity of an active ID. Unknown IDs are rejected without changing the book.
Amendment	Amend an active limit order by specifying new price and NEW REMAINING quantity. Same price and a decrease preserve priority. Same price and equal quantity is a successful no-op. Increase or price change loses priority and may match.
Validation	Reject zero quantities, bad enums, invalid prices, duplicates, and unknown targets before any matching or removal. A rejected command leaves book state and trade output unchanged.

Quantity zero is not a cancellation shortcut. Use Cancel. Side cannot change through Amend. Market commands carry no meaningful limit price; use a separate command type or require a documented zero placeholder.

Results ownership and failure behavior

Define the public result before the loop

A command result contains success or a rejection reason, a vector of trades in execution order, resting residual, and expired residual. Cancel and Amend also report the affected remaining quantity. Each Trade contains maker ID, taker ID, resting price, and executed quantity. A successful operation can produce zero trades.

For a New command, original quantity equals executed quantity plus resting residual plus expired residual. Count each trade once when checking the aggressor. When checking both sides together, executed volume removes that quantity from each side. For Amend, treat a quantity increase or reduction as an explicit adjustment to the ledger; do not apply the original New equation unchanged.

Make sequence and ownership explicit

Assign a command sequence before processing every command, including rejected ones. Record that sequence in results. Assign a fresh arrival priority only when an order joins a FIFO queue or loses priority. A quantity decrease and a no-op amendment keep their old priority. Never use wall-clock timestamps to decide matching order.

OrderBook owns resting orders and price levels. The active-ID table is an index into that ownership. Return value snapshots for tests; do not expose mutable list iterators. Once you store internal iterators, disable copying until you can rebuild the index for a copy. Keep movement disabled as well until container and resource lifetimes have been reviewed.

Choose a bounded arithmetic policy

Use checked addition for aggregate quantities: before $\text{total} + q$, require $q \leq \text{UINT64_MAX} - \text{total}$. Keep configuration limits for live orders and pending output. For the initial growable STL implementation, treat allocation failure as fatal for that engine instance. Do not catch `bad_alloc` and continue with potentially partial mutations. This version promises unchanged state for ordinary validation rejection, not rollback after arbitrary allocation failure.

When P02 introduces fixed capacity, reject capacity exhaustion before mutation. An accepted sweep can create many trades; preflight or reserve output capacity before applying its mutations. State the same failure policy for an amendment that loses priority.

Establish a small architecture

Component	Responsibility
OrderBook	Own state, match commands, maintain invariants.
MatchingEngine	Assign command sequence and return results; keep this thin for one symbol.
ReferenceBook	Implement the same contract with independent, slow logic.
Replay runner	Read commands, invoke the engine, compare or serialize results.
CLI and later gateway	Parse input and display or transmit results.

Start with concrete classes and ordinary structs. Type aliases improve names but do not create distinct strong types. Add wrapper types only when they prevent a concrete mix-up. Keep matching code free of console output, sockets, and benchmark clocks.

Native Windows with MSYS2 UCRT64 g++

Use your existing MSYS2 installation if it works. This workbook uses the UCRT64 g++ compiler, CMake, Ninja, GDB, and VS Code. The executables are native Windows programs. The installation commands use an MSYS2 shell; all project build commands use PowerShell with the UCRT64 tools on PATH. No WSL is involved.

Install the matching tool family

Open MSYS2 UCRT64 from the Start menu. Update the installation with `pacman -Syu`. If the update asks to close terminals, follow it, reopen UCRT64, and repeat the update. MSYS2 supports complete system upgrades. Then install these packages in that UCRT64 shell. [1]

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc \
  mingw-w64-ucrt-x86_64-cmake \
  mingw-w64-ucrt-x86_64-ninja \
  mingw-w64-ucrt-x86_64-gdb
```

The package prefix matters: use the UCRT64 family throughout. Do not substitute the MSYS `/usr/bin/g++` or mix MINGW64 and UCRT64 library paths. The default installation is `C:\msys64`; replace that root throughout if yours differs. Record installed package versions with `pacman -Q` in the MSYS2 terminal. [1]

Build from PowerShell

Open PowerShell at `C:\dev\matching-engine`. Prepend the tools directory for that terminal session, then check every executable. `Get-Command` should resolve these tools under `C:\msys64\ucrt64\bin`. Start VS Code from this prepared shell so its child processes inherit PATH. [3]

```
$env:Path = "C:\msys64\ucrt64\bin;$env:Path"
Get-Command g++, cmake, ninja, gdb
g++ --version
g++ -dumpmachine
cmake --version
ninja --version
gdb --version
code .
```

Expect an x86_64-w64-mingw32 target. Record the actual compiler version instead of assuming a particular rolling-release version. The same PATH lets built programs find the toolchain's runtime DLLs. An executable that works in this terminal but fails from Explorer may be missing those DLLs on Explorer's PATH.

Follow a file through the build

A header declares an interface. A .cpp defines behavior. Compilation turns each translation unit into an object; linking combines objects and resolves calls into libraries. CMake describes dependencies and creates build instructions; Ninja executes them. A header-not-found error points to include paths. An undefined-reference error points to definitions or linkage. Adding a header path does not supply a missing function body.

Configure the Microsoft C/C++ extension with compilerPath C:/msys64/ucrt64/bin/g++.exe, IntelliSense mode windows-gcc-x64, and cppStandard c++23. Use the CMake build commands in L02 rather than the editor's single-file build button: the project has multiple sources and a test library.

Use the matching debugger

After building L02, save this as .vscode/launch.json. Build Debug before pressing F5, put a breakpoint in build_probe, then step into it. If GDB cannot start the program, verify the program path and the inherited UCRT64 runtime PATH. [3]

```
{
  "version": "0.2.0",
  "configurations": [{
    "name": "Matching engine g++ Debug",
    "type": "cppdbg",
    "request": "launch",
    "program": "${workspaceFolder}/build/debug/lob_simulator.exe",
    "cwd": "${workspaceFolder}",
    "MIMode": "gdb",
    "miDebuggerPath": "C:/msys64/ucrt64/bin/gdb.exe",
    "stopAtEntry": false
  }]
}
```

Exercise: CMake's cached compiler is cl.exe after you change the editor to g++. Answer: changing IntelliSense does not change the build. Configure a fresh build directory with g++; never reuse the old MSVC build cache. Use the Debug, Checked, and Release directories in the included starter commands.

L01 Set up the Windows g++ toolchain

Prerequisite: none. Read the preceding L01 teaching pages first.

Understand the mechanism

g++ compiles sources and invokes the linker. CMake describes targets and generates a Ninja build. GDB inspects the running executable. The UCRT64 tool family produces native Windows programs and supplies matching headers, libraries, and runtime DLLs.

Work through an example

If VS Code uses one compiler for diagnostics while CMake cached another, the editor and build can disagree. Get-Command and CMake's configuration log establish which programs actually run. A compiler change requires a fresh build directory.

Build it in this order

1. Update MSYS2 and install the four UCRT64 packages using the included L01 commands. Reuse your existing installation if it passes these checks.
2. Prepare PowerShell's PATH. Verify g++, cmake, ninja, and gdb all resolve under the UCRT64 bin directory. Record g++ --version and g++ -dumpmachine.
3. Launch VS Code from that shell. Select the UCRT64 g++.exe for IntelliSense and use the included GDB launch configuration after L02 builds.
4. Record tool versions, target, and Windows version in docs/environment.md. Keep MSVC, MSYS, and other MinGW build caches separate from this project.

Check the result

You can locate all tools, explain compile versus link, and identify CMakeCache.txt. You can explain why the UCRT64 runtime PATH matters when launching the executable.

If it fails: check the installation root and package prefix. For mixed-header errors, verify compilerPath and remove manually added paths from another toolchain. Start a fresh build directory after a compiler change.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Understand the scaffold and the first test

The following practice page and three starter pages contain all the files and commands for this lesson. This reading explains why each part exists, so you can extend the scaffold without treating CMake as an unexplained template.

Targets express dependencies

CTest is CMake's test runner; GoogleTest is the library that supplies the test cases and assertions. CMake's `include(CTest)` enables test registration and its `BUILD_TESTING` option. `FetchContent` downloads/builds the pinned dependency. `GTest::gtest_main` supplies the test executable's main function, which starts the tests; you do not write a second main in `smoke_test.cpp`. `find_package(Threads)` supplies the platform's thread-linking requirements through `Threads::Threads`.

`add_library(lob_core src/order_book.cpp)` creates the compiled core. `target_include_directories(lob_core PUBLIC include/lob)` makes `include/lob` headers available when building the library and its consumers. `target_link_libraries(lob_simulator PRIVATE lob_core)` makes the executable depend on that library. `PUBLIC` exports a usage requirement; `PRIVATE` applies it to the named target. An `INTERFACE` library such as `lob_options` carries options without compiling a .cpp of its own. [2]

The simulator and tests share `lob_core`. Later, add source files explicitly to its source list. Add each new test .cpp to `lob_tests`. Merely placing a file in the directory does not add it to these targets. CMake generates build instructions; `cmake --build` invokes the selected build tool; CTest starts registered test commands.

Write assertions that diagnose a failure

GoogleTest discovers `TEST(Suite, Name)` cases. `EXPECT_EQ` reports a mismatch and continues within that test. `ASSERT_EQ` stops the current test function when the condition fails. Use an assertion before indexing a result whose size is part of the expectation. [4]

```
// Pattern for later matching tests, not a complete API definition.
const auto result = book.submit(command);
ASSERT_EQ(result.trades.size(), 1u);
EXPECT_EQ(result.trades[0].maker_id, 10u);
EXPECT_EQ(result.trades[0].price, 101);
EXPECT_EQ(result.trades[0].quantity, 5u);
```

Checking `trades[0]` before checking the size turns a useful test failure into an invalid memory access. Keep expected IDs, prices, and quantities literal in hand-worked cases. Reusing the matching helper to calculate the expected answer can duplicate its bug.

Interpret the three configurations

Debug favors GDB inspection. Release enables optimization and commonly defines `NDEBUG`, which removes ordinary assert checks. GoogleTest expectations still run. Checked is a separate Debug build with `libstdc++` debug mode: `-D_GLIBCXX_DEBUG` and `-D_GLIBCXX_DEBUG_PEDANTIC`. It detects many invalid container and iterator operations. It does not provide general heap memory instrumentation. [5]

libstdc++ debug mode changes the representation of standard containers. Compile linked code that exchanges those containers with consistent settings, including GoogleTest and your core. The starter sets the definitions before adding any target or fetching GoogleTest. Never link a prebuilt unchecked library across such a container boundary in Checked. Release must not inherit those definitions. [5]

AddressSanitizer (ASan), UndefinedBehaviorSanitizer (UBSan), and ThreadSanitizer (TSan) diagnose different classes of memory, language-rule, and race errors. The required path makes no claim to have run them. Runtime support depends on the compiler target; do not assume Linux-style -fsanitize=address,undefined recipes work in a native MinGW-w64 build. This Windows g++ path uses checked containers, GDB, explicit bounds checks, invariants, and differential tests. These checks have narrower coverage than a memory sanitizer, which remains outside the required toolchain. [6]

Read the flags before running them

In CMake commands, -S selects the source directory, -B the build directory, -G the generator, and -D sets a named configuration variable. --build builds generated targets; CTest's --test-dir selects a configured test directory. --output-on-failure prints diagnostics for failing tests, -N lists tests without running them, and --verbose shows executed build commands.

The starter's g++ flags are diagnostics: -Wall enables common warnings; -Wextra adds more; -Wpedantic diagnoses some nonstandard extensions; -Wconversion reports potentially value-changing implicit conversions; -Wshadow reports names hiding other names. CMake selects the Debug/Release optimization and debug-information flags. Treat warnings as questions to resolve, rather than suppressing them by copying unrelated flags.

Verify what actually ran

```
ctest --test-dir build/debug -N
ctest --test-dir build/debug --output-on-failure
cmake --build build/debug --verbose
```

The first command lists registered tests. The second must execute them. The third exposes compilation and linking commands when you need to inspect flags or include paths. A successful CTest command that ran zero tests is an incomplete setup.

The first configure fetches the pinned GoogleTest archive. You need an internet connection for that dependency and for tool installation; no external reading is needed to understand the exercises. After fetching, keep the dependency cache while working offline. Preserve the release pin and record the archive hash if you need byte-for-byte dependency identification.

Exercise and answer

Change the starter expectation from 23 to 24. Predict which steps still succeed. Answer: compilation and linkage still succeed; CTest fails when the executable compares 23 with 24. Restore 23 and record both outcomes. If you remove order_book.cpp from lob_core instead, expect a build configuration or linkage failure before that assertion can run. Complete the practice page, then follow the included starter pages in order.

L02 Build a library and one test

Prerequisite: L01. Read the preceding L02 teaching, then the three starter pages that follow this practice page.

Understand the mechanism

The simulator and tests link the same `lob_core` library. CTest launches GoogleTest cases so a broken expectation produces a nonzero test result. Begin with a build probe before introducing matching logic.

Work through an example

The library returns 23. The simulator prints 23 and the test expects 23. Changing the expectation to 24 must make CTest fail while compilation and linking still succeed.

Build it in this order

1. Create the four starter files and CMakeLists.txt on the following pages. Configure Debug with Ninja and g++, build, and run CTest.
2. Make the test fail on purpose, inspect the result, restore it, and rerun. In GDB, break inside `build_probe` and step into it from the simulator.
3. Configure the separate Checked build. All targets, including GoogleTest, must use consistent `libstdc++` debug-container settings. Then run Release tests too.
4. Commit the scaffold, ignore `build/`, and document the exact commands. Explain each CMake target and what PUBLIC exposes to consumers.

Check the result

CTest lists and runs one passing test after restoration. The deliberately wrong expectation fails. GDB reaches the library source. Debug, Checked, and Release binaries each launch from the prepared PowerShell.

If it fails: zero registered tests is not completion. An undefined reference suggests a missing source or library dependency. A runtime DLL error suggests a missing UCRT64 bin directory on PATH. Do not mix Checked and ordinary container ABIs across targets.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Starter files for the first test

Create these files under a project directory such as C:\dev\matching-engine. This bootstrap exercises library linkage and testing. The function is a build probe, not the future matching API. Replace it after L02.

Header and library source

include/lob/build_probe.hpp

```
#pragma once
namespace lob {
    int build_probe();
}
```

src/order_book.cpp

```
#include "lob/build_probe.hpp"
namespace lob {
    int build_probe() { return 23; }
}
```

Executable and test

apps/simulator.cpp

```
#include "lob/build_probe.hpp"
#include <iostream>
int main() {
    std::cout << lob::build_probe() << '\n';
}
```

tests/smoke_test.cpp

```
#include "lob/build_probe.hpp"
#include <gtest/gtest.h>
TEST(BuildProbe, LinksCoreLibrary) {
    EXPECT_EQ(lob::build_probe(), 23);
}
```

The expected simulator output is 23. The expected test count is one passing test. Change 23 to 24 in the test, confirm failure, and restore it. GoogleTest assertions remain active in optimized test builds; ordinary assert expressions may be compiled out. [4]

First directory layout

CMakeLists.txt lives at the repository root. Put public headers in include/lob, implementation in src, test sources in tests, and the CLI in apps. Add benchmarks and docs when their lessons need them. Ignore the build directory in Git.

CMake configuration for Windows g++

Save as CMakeLists.txt at the project root. Use the UCRT64 CMake and Ninja packages. C++23 is requested explicitly; record the g++ version that builds the project. Checked definitions precede all targets, including GoogleTest. [2, 4, 5]

```
cmake_minimum_required(VERSION 3.28)
project(lob LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 23)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
if(NOT CMAKE_CXX_COMPILER_ID STREQUAL "GNU" OR NOT MINGW)
    message(FATAL_ERROR "Use native Windows MSYS2 UCRT64 g++")
endif()
option(LOB_CHECKED "Use libstdc++ checked containers" OFF)
if(LOB_CHECKED)
    if(NOT CMAKE_BUILD_TYPE STREQUAL "Debug")
        message(FATAL_ERROR "Use Debug for the Checked build")
    endif()
    add_compile_definitions(_GLIBCXX_DEBUG _GLIBCXX_DEBUG_PEDANTIC)
endif()
add_library(lob_options INTERFACE)
target_compile_options(lob_options INTERFACE
    -Wall -Wextra -Wpedantic -Wconversion -Wshadow)
find_package(Threads REQUIRED)
add_library(lob_core src/order_book.cpp)
target_include_directories(lob_core PUBLIC include)
target_link_libraries(lob_core PUBLIC lob_options Threads::Threads)
add_executable(lob_simulator apps/simulator.cpp)
target_link_libraries(lob_simulator PRIVATE lob_core)
include(CTest)
if(BUILD_TESTING)
    include(FetchContent)
    set(BUILD_GMOCK OFF CACHE BOOL "" FORCE)
    FetchContent_Declare(googletest
        URL https://github.com/google/googletest/archive/refs/tags/v1.17.0.zip
        DOWNLOAD_EXTRACT_TIMESTAMP TRUE)
    FetchContent_MakeAvailable(googletest)
    add_executable(lob_tests tests/smoke_test.cpp)
    target_link_libraries(lob_tests PRIVATE lob_core GTest::gtest_main)
    include(GoogleTest)
    gtest_discover_tests(lob_tests)
endif()
```

Add new .cpp files to their targets. Link later networking targets with ws2_32. The compiler guard checks GNU/MinGW; the L01 path check establishes UCRT64. Do not reuse an old MSVC build directory.

Configure, build, test, and debug

Run at the project root in PowerShell. Change the MSYS2 root if yours differs. All three builds use the same UCRT64 toolchain. The first configure downloads the pinned GoogleTest source; subsequent builds reuse it.

```
$env:Path = "C:\msys64\ucrt64\bin;$env:Path"
Get-Command g++, cmake, ninja, gdb
```

Debug

```
cmake -S . -B build/debug -G Ninja `
  -DCMAKE_CXX_COMPILER=g++ -DCMAKE_BUILD_TYPE=Debug
cmake --build build/debug
ctest --test-dir build/debug --output-on-failure
.\build\debug\lob_simulator.exe
```

Checked containers

```
cmake -S . -B build/checked -G Ninja `
  -DCMAKE_CXX_COMPILER=g++ -DCMAKE_BUILD_TYPE=Debug `
  -DLOB_CHECKED=ON
cmake --build build/checked
ctest --test-dir build/checked --output-on-failure
```

Release and release tests

```
cmake -S . -B build/release -G Ninja `
  -DCMAKE_CXX_COMPILER=g++ -DCMAKE_BUILD_TYPE=Release
cmake --build build/release
ctest --test-dir build/release --output-on-failure
```

A PowerShell continuation backtick must be the last character on its line. Use L01's VS Code GDB configuration or run `gdb .\build\debug\lob_simulator.exe` and enter `break lob::build_probe`, `run`, `backtrace`, and `quit`. Use `cmake --build build/debug --verbose` to inspect flags and linkage. Run benchmarks only from Release with `LOB_CHECKED` off.

Orders, prices, and ownership

A bid is an offer to buy. An ask is an offer to sell. The best bid is the highest buy price; the best ask is the lowest sell price. Their pair is the best bid and offer, or BBO. Depth sums remaining quantity at each price. None of those summaries identifies which individual order executes first.

A level is an ordered collection of claims

Suppose bids arrive as ID 1: price 100, quantity 10; ID 2: price 100, quantity 20; ID 3: price 99, quantity 40. At 100, depth is 30, but FIFO is [1, 2]. An incoming sell for 12 should consume all 10 from ID 1 and then 2 from ID 2. Replacing the queue with a single quantity of 30 loses the maker IDs and priority information.

Use integer ticks instead of floating-point money. Here 101 means tick 101; it does not imply \$1.01. The contract sets price and quantity ranges. A market command has no limit, while a limit command can execute only at its limit or better. The simulator's resting-price rule is a declared project policy. [9]

Choose one owner for each object

The map owns its levels and each level owns its orders. A snapshot owns copies for inspection. Returning a value is safe even when the engine later changes. Returning a reference or iterator lets the caller outlive, erase, or mutate something the book owns. Keep that boundary narrow from the start.

```
using Price = std::int64_t;
using Quantity = std::uint64_t;
using OrderId = std::uint64_t;
enum class Side { Buy, Sell };

struct Order {
    OrderId id;
    Side side;
    Price price;
    Quantity remaining;
    std::uint64_t priority;
};
// Baseline storage; include <map>, <deque>, <functional>, <cstdint>.
std::map<Price, std::deque<Order>, std::greater<Price>> bids;
std::map<Price, std::deque<Order>> asks;
```

Both maps expose their best occupied price at `begin()`. Check `empty()` before dereferencing it. Use `std::optional<Price>` for a missing best price instead of inventing a sentinel such as zero or the largest integer.

Derive the snapshot before coding it

Choose one canonical order: bids first in descending price, asks next in ascending price, then FIFO within each level. Include ID, side, price, remaining quantity, and priority. Tests compare this value sequence. An unordered ID index must never determine snapshot order.

Exercise: add ID 4 at bid 101 after IDs 1 and 2. Which maker should a sell visit first? Answer: 4. Price priority outranks earlier arrival at a worse price. At 100, ID 1 still precedes ID 2. Canceling either order must preserve the other's identity, which is why order-level storage matters.

L03 Represent the passive order book

Prerequisite: matching contract and L02. Suggested work unit: 60 to 120 minutes.

Understand the mechanism

A price level contains a FIFO queue of separate orders. An ordered map selects the best occupied price; the queue determines which order at that price executes first. Use descending bids and ascending asks. The level owns its orders. For this first version, deque gives simple append and front removal, while cancellation will scan.

Work through an example

Insert bids 1 at 100 for 10 units, 2 at 100 for 20, and 3 at 99 for 40. Insert ask 4 at 102 for 10. Best bid is 100 with aggregate quantity 30; best ask is 102 with quantity 10. The bid queue at 100 is [1, 2], not one order of quantity 30.

Build it in this order

1. Define Side, Price, Quantity, OrderId, Order, and Trade in namespace lob. Use ordinary aliases at first and enforce the contract through validation functions.
2. Create OrderBook with `map<Price, deque<Order>, greater<Price>>` bids and an ascending ask map. Return optional values for an empty best bid or ask.
3. Implement a passive insertion helper and a value snapshot ordered by side, price, and FIFO position. Keep this helper internal once matching is available.
4. Add tests for empty state, multiple orders at one price, opposite-side separation, and aggregated depth. Compare explicit expected values; do not calculate the expectation with the production helper.

Check the result

The four-order example has the stated BBO, depth, and FIFO order. Changing the bid comparator makes a test fail. Invalid input is rejected before the passive helper mutates state.

If it fails: A snapshot that looks sorted can conceal reversed queue order. Inspect individual IDs as well as aggregate quantities. Never return a reference to a temporary snapshot.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☒ The completion checks pass.

Evidence or commit: Actual minutes:

Derive a single execution

Treat matching as a state transition with a result, rather than a loop that prints trades. Input validation, crossing, arithmetic, and cleanup have separate jobs. That separation makes rejection tests and later replay possible.

Conditions before arithmetic

For a buy limit L and best ask A , cross when $L \geq A$. For a sell limit L and best bid B , cross when $L \leq B$. If either side lacks a maker, no execution occurs. For a valid crossing pair, $q = \min(\text{maker.remaining}, \text{taker.remaining})$. Therefore q is no larger than either operand, which proves both subtractions are safe when the operands were valid.

Maker quantity	Taker quantity	Execution	What remains
8	5	5	Maker 3; taker 0
5	8	5	Maker 0; taker 3
8	8	8	Neither order
8	5, no cross	0	Maker 8; limit taker rests 5

The execution price is the maker's price. A buy limit of 103 meeting an ask at 101 pays 101 in this simulator. The limit grants permission to execute up to 103; it does not replace the resting price.

Apply effects in a safe order

Read the maker ID and price before erasing anything. Construct the trade using those values and q . Subtract q from both remaining quantities and any cached level total. If the maker reaches zero, remove its active-ID entry once the index exists, erase its node, and remove an empty level. Do not read the erased maker afterward. The growable implementation treats allocation failure as fatal to that instance, as stated in the contract.

The incoming order is a local value during matching. It becomes owned by the resting book only if a limit residual remains at the end. This prevents an aggressor from appearing in the active index while it is still executing. Duplicate-ID validation still happens before any fills, even for an order that will execute in full.

Separate conservation from trade count

For a new quantity Q , sum all executed quantities once for the taker. Then $Q = \text{executed} + \text{resting} + \text{expired}$. A trade of 5 removes 5 from each side, but the taker's executed quantity is 5, not 10. Cancel emits no trade; an amendment needs an explicit quantity adjustment.

Exercise: ask 10 has 8 at 101. A buy 20 has 5 at limit 100. Answer: no trade; ask 10 stays at 101 for 8 and bid 20 rests at 100 for 5. Changing the limit to 103 produces one trade at 101 for 5 and leaves only ask 10 for 3. Use both cases as exact assertions before extracting shared buy/sell code.

L04 Match one resting order

Prerequisite: L03. Suggested work unit: one or two focused sessions.

Understand the mechanism

The incoming order is the aggressor, or taker; the resting order is the maker. A limit allows execution at its price or better. Under this contract, the resting order sets the execution price. The matched quantity is $\min(\text{maker_remaining}, \text{taker_remaining})$. Decide the crossing condition before changing either quantity.

Work through an example

Rest sell 10 at 101 for 8. Submit buy 20 at 103 for 5. Produce one trade, maker 10, taker 20, quantity 5, price 101. The ask keeps 3; the buyer has no residual. Reversing the quantities leaves a buy residual of 3, which rests at its own limit of 103.

Build it in this order

1. Define the command result and rejection reasons from the contract. Validate the incoming ID, quantity, side, and limit before matching.
2. Implement buy against one best ask. Handle no cross, equal quantities, partial maker fill, and partial taker fill in separate tests.
3. Subtract the matched quantity only after computing its minimum. Remove a completely filled maker and its empty level; rest a limit residual at the incoming limit.
4. Add the symmetric sell case. Extract common mechanics only after both directions pass, keeping the crossing and best-price choices readable.

Check the result

All four quantity cases pass on both sides. The trade price uses the maker price even when the incoming limit is better. For each New, executed + resting + expired equals the original quantity.

If it fails: If a full maker remains with zero quantity, cleanup is incomplete. If the incoming order vanishes with unmatched units, residual handling is missing. Check unsigned subtraction and the chosen trade price before refactoring.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Prove the sweep and residual rules

The one-maker transition is the unit of work. The sweep repeatedly applies it to the best eligible maker. Every iteration either finishes the taker or removes a maker, so a finite book cannot produce an infinite matching loop when quantities stay positive.

Readable control flow

```
validate the complete command without changing the book
while taker.remaining > 0:
    if opposite side is empty: break
    locate its best level
    if limit command does not cross that price: break
    locate the oldest maker at that level
    execute min(maker.remaining, taker.remaining)
    remove a filled maker and, if empty, its level
if taker.remaining > 0:
    limit: append residual at its own price
    market: report residual as expired
```

Recheck price on every iteration. A cross at 101 says nothing about eligibility at 103. Reacquire locations after cleanup. A reference to a deleted level is invalid even if the next map node happens to occupy nearby memory.

Hand-work the complete sweep

Step	Best eligible maker	Fill	Taker left	Maker left
Start	Buy 20 has 17, limit 102	0	17	-
1	Sell 10: 4 at 101	4 at 101	13	0
2	Sell 11: 6 at 101	6 at 101	7	0
3	Sell 12: 10 at 102	7 at 102	0	3

Sell 13 at 103 for 20 remains untouched. The result must list makers [10, 11, 12] in that order. Aggregate executed volume is 17. A later market buy for 25 takes 3 at 102 and 20 at 103, then expires 2. It never rests at an artificial infinite price.

Why the resting book is uncrossed

Assume the initial resting book is uncrossed. A new limit residual rests only after either the opposite side empties or its best price fails the crossing condition. All worse opposite prices also fail that condition. Canceling or reducing an order cannot introduce a cross. A priority-losing amendment goes through matching again. These facts preserve an uncrossed resting book after every command.

Exercise: replace buy 20's quantity with 25 and keep its limit at 102. Answer: fills 4, 6, and 10, totaling 20; residual 5 rests at bid 102; ask 13 at 103 stays unchanged. A market buy for 25 from the original book instead takes another 5 at 103 and leaves ask 13 with 15. This pair of tests catches accidental market-like behavior in limit sweeps.

L05 Sweep price levels and handle markets

Prerequisite: L04. Suggested work unit: one or two focused sessions.

Understand the mechanism

Matching is a repeated local step. While the aggressor has quantity, select the best opposite price, stop if a limit no longer crosses, and consume the oldest maker there. Deleting a level changes where the next maker lives. A market order has no limit barrier and expires any residual after the opposite book empties.

Work through an example

Rest asks 10 at 101 for 4, 11 at 101 for 6, 12 at 102 for 10, and 13 at 103 for 20. A buy 20 at 102 for 17 executes 4 against 10 at 101, 6 against 11 at 101, and 7 against 12 at 102. Remaining asks are 12 with 3 at 102 and 13 with 20 at 103.

Build it in this order

1. Turn the one-maker operation into a loop. Reacquire the next level or order after erasing; never increment an iterator you just invalidated.
2. Write the worked sweep as a test with exact maker IDs, quantities, prices, and trade order. Add a better-price order that arrived later to prove price outranks arrival.
3. Add a separate Market command or an explicit order-kind field. A market buy of 25 against the example's remaining book executes 3 at 102 and 20 at 103, then expires 2.
4. Test a market into an empty book, a non-crossing limit, a one-sided book, and multiple FIFO makers. Check invariants after every command, even in failed-command tests.

Check the result

The sweep matches the exact three trades. The later market has executed 23, resting 0, expired 2. No successful command leaves a crossed resting book, an empty level, or a zero-quantity resting order.

If it fails: Stopping after the first fill loses sweeps. Checking price once before the loop can execute beyond a limit. A market residual in the resting snapshot violates the contract.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Invariants describe a legal state

A scenario test checks one expected outcome. An invariant checks a property that must hold after every processed command. Both are needed: an internally consistent book can still choose the wrong maker, and a correct trade list can conceal a stale node or total.

Recompute rather than trusting cached state

Walk the book after each command in tests. Build a fresh set of observed IDs and a fresh sum for every level. Check the following properties from the actual orders.

Property	How to check it
No empty levels	Every stored FIFO has at least one order
Valid order	ID, price, and remaining size satisfy the contract
Correct location	Stored side and price equal the owning side and map key
Unique active ID	Inserting the ID into a temporary set succeeds once
FIFO	Arrival priorities increase along each queue
Quantity total	Checked sum of order quantities equals cached depth
No cross	If both sides exist, best bid < best ask
Later ID index	Observed IDs and locator keys are exactly equal

The checker may be slow. Keep it callable from Release tests; do not hide its entire body behind assert. It is not part of a headline performance loop unless you say so.

Start cancellation with a scan

Search both sides, then each level's queue. Once the target is found, save the removed quantity, erase it, update totals, remove an empty level, and return success. Never increment a container iterator after erasing the object it identified. A scan may visit every active order, giving $O(N)$ search cost. That is a useful baseline for the next lesson.

For unknown IDs, compare the entire before/after snapshot and require an empty trade list. The engine command sequence still advances for a validly submitted but rejected command. 'No book mutation' does not mean 'no result or sequence number.'

Track the lifecycle of one ID

ID 7 rests, partly fills, is canceled, and may then be reused. A new command with ID 7 while it remains active is a duplicate, including a market command that would otherwise execute at once. A cancel after its full fill is unknown. These cases pin down the active-only uniqueness rule.

Exercise: FIFO [1:4, 2:6, 3:8] at bid 100; cancel 2, then sell 5 at 100. Answer: cancel removes 6 without a trade; the sell executes 4 against 1 and 1 against 3; only 3 remains with 7. A later cancel 2 must reject. Checking depth alone would miss an incorrect trade against maker 2.

L06 Cancel by ID and formalize invariants

Prerequisite: L05. Suggested work unit: one or two focused sessions.

Understand the mechanism

Cancellation removes an order without a trade. The baseline can scan levels and orders until it finds an ID. This is $O(N)$ in active orders and is acceptable as a reference point. Deleting the last order removes the level. Rejected cancellations must leave the same canonical snapshot and output.

Work through an example

At bid 100, the FIFO is [1 with 4, 2 with 6, 3 with 8]. Cancel 2. The queue becomes [1, 3] and depth becomes 12. A later sell of 5 fills 1 for 4 and 3 for 1. Cancel 2 again: return UnknownOrder with no state change.

Build it in this order

1. Implement scan cancellation and reject an unknown ID. Check duplicate IDs by active-order membership, including a fully crossing New: it cannot bypass duplicate validation.
2. Create `validate_invariants()` for tests. Check nonempty levels, positive quantities, unique active IDs, correct side and price, FIFO priority, checked aggregate totals, and an uncrossed resting book.
3. Test head, middle, and tail cancellation, cancellation after partial and complete fills, and deleting the last order at a level.
4. Confirm ID reuse after complete fill or cancel succeeds. Save a short command vector that combines insertion, a sweep, rejection, and cancellation. It will become a replay fixture in L11.

Check the result

Every scenario passes the invariant checker after each command. A rejected cancellation produces no trade or book mutation. You can explain why the scan may inspect all active orders.

If it fails: Avoid storing deque iterators in a persistent index at this stage. Middle erasure can invalidate locators. Recompute aggregate expectations from the snapshot rather than trusting a cached total.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Stable locators without dangling handles

An iterator identifies an element owned by a container. Its validity depends on the exact operation on that container, not on whether the iterator object still exists. A dangling iterator can look plausible and compile without complaint.

The operation rules you need

Storage operation	Consequence for this project
list insertion	Existing element iterators remain valid
list erase of one node	Only handles to that erased node become invalid
map insertion	Existing element iterators remain valid
map erase of one level	Handles into that level and its owned orders die
unordered_map rehash	Its iterators become invalid; do not cache them across insertion
deque middle erase	Can invalidate other locations; unsuitable for these persistent locators

The list and map rules support storing a level iterator plus an order iterator. An iterator into the ID hash table itself has a different lifetime from the list iterator stored as its value. Read those as two separate relationships. [10]

Run a complete miniature first

```
#include <cassert>
#include <iterator>
#include <list>
#include <unordered_map>

int main() {
    std::list<int> fifo{1, 2, 3};
    std::unordered_map<int, std::list<int>::iterator> by_id;
    for (auto it = fifo.begin(); it != fifo.end(); ++it)
        by_id.emplace(*it, it);

    const auto node = by_id.at(2);
    by_id.erase(2);
    fifo.erase(node);
    assert(*by_id.at(1) == 1);
    assert(*by_id.at(3) == 3);
    assert(!by_id.contains(2));
}
```

Compile this exercise without NDEBUD, or replace assert with GoogleTest expectations. Do not dereference node after erase. A failed lookup for 2 is the correct behavior. The ID index does not own the list node and cannot extend its lifetime.

Integrate two levels of location

Use separate bid and ask locator structs, or a variant containing them, because comparator-specific map types can yield different iterator types. Each locator holds its owning level iterator and the list iterator. The side selects the correct map. Disable copy and move for the first indexed OrderBook. A memberwise copy of an index would continue pointing into the original book.

For cancellation: find the ID entry; copy the locator and removed quantity; erase the ID entry; erase the list node; erase the map level if it is empty. For a full maker fill, perform the same cleanup through one reviewed helper. For a reduction, retain the node and locator. For a priority-losing amendment, remove the old locator and create a new one only if residual rests.

Hash lookup is expected $O(1)$, not worst-case $O(1)$. List erasure is constant time; map erase by iterator is amortized constant. If you store only price, map.find adds $O(\log P)$, where P is occupied levels. A sweep visiting K makers still costs at least $O(K)$, including its outputs. [10]

Exercise: the ID table rehashes after a new order. Does that invalidate the list iterators stored inside its values? Answer: rehash invalidates hash-table iterators, not nodes in a separate list. But erasing the owning price level destroys its list and all list nodes. Your invariants must verify ownership as well as ID equality.

L07 Use stable locations for cancellation

Prerequisite: L06. Suggested work unit: one or two focused sessions.

Understand the mechanism

A locator stores where an active order lives. Change each FIFO to `list<Order>`; erasing one node leaves other list iterators valid. The map and lists own the nodes, while `unordered_map<OrderId, Locator>` indexes them. Store the list iterator and the owning level location. A price-only locator still needs an $O(\log P)$ map lookup.

Work through an example

Create one list containing IDs 1, 2, 3 and an ID-to-iterator table. Erase ID 2 through its stored iterator, then remove that table entry. Iterators for 1 and 3 still locate their nodes. Looking up 2 must now fail. Copying this index into a copied book would still point at the original nodes unless you rebuilt it.

Build it in this order

1. Do the three-node exercise outside the book. Before erasing, copy any ID or level information you will need afterward.
2. Switch FIFO storage to list and rerun existing tests. Only then add the active-ID index. Insert an index entry only for an order that rests; erase it on full fill and cancel.
3. Store the owning level iterator with its side. Bid and ask map iterator types can differ, so use a small variant or separate side-specific locator structs. Disable OrderBook copy and move for this stage.
4. Check that index keys equal resting IDs and every locator round-trips to its own ID. Compare scan and indexed cancellation on the same prebuilt books; exclude setup from the timing.

Check the result

Cancellation performs no order scan. With direct level and list iterators, hash lookup is expected $O(1)$, list erase is $O(1)$, and map erase by iterator is amortized $O(1)$. This is not a worst-case constant-time claim. A price lookup adds $O(\log P)$. [10]

If it fails: After a fill, stale index entries often survive. After a final-node cancel, level cleanup may use an erased iterator. Never inspect a dead node to discover its ID.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Amendment is a transaction with priority rules

Do not begin an amendment by canceling the old order. First locate and validate it, determine the transition, and prepare resources according to the current failure policy. Only then mutate. Ordinary rejection preserves the original quantity, location, and FIFO position.

Decide from current remaining quantity

Requested change	Effect on old node	Matching and priority
Unknown ID or invalid field	None	Reject
Same price, same remaining	None	Successful no-op
Same price, lower remaining	Reduce in place	Preserve priority
Same price, higher remaining	Remove old residual	Resubmit; lose priority
Different valid price	Remove old residual	Resubmit; lose priority

Suppose the original quantity was 10 and 4 has filled. Current remaining is 6. Amending to 8 means 8 remaining, a size increase of 2. It does not mean a total size of 8 with only 4 left. This choice belongs to the workbook contract, regardless of a venue's terminology.

Separate public validation from internal resubmission

A public New checks for an active duplicate ID. An amendment already owns its target ID. Validate it first, remove its old active entry when the transition loses priority, then use an internal validated path for the new attempt. Do not bypass all validation for arbitrary callers just to avoid rejecting the amendment's own ID.

For a price change, the resubmitted order keeps its side and ID, matches against the opposite book, and appends any residual at the new price with a fresh priority. If it fills in full, no active entry remains.

Reserve/preflight bounded output before removal when the fixed-capacity variant exists. The earlier growable variant still has fatal-instance behavior on allocation failure.

Use a quantity-adjustment ledger

If remaining changes from 6 to 8, record an adjustment of +2. If it changes from 6 to 3, record -3. That explains the change in potential execution volume without pretending the original new-order conservation equation remained unchanged. It also supplies a clear cancellation/amendment response.

Exercise: bids [1:6, 2:5] at 100. Amend 1 to 8 at 100, then submit sell 3 for 7 at 100. Answer: FIFO becomes [2:5, 1:8]; sell 3 fills 5 against 2 and 2 against 1; 1 remains with 6. If the amendment instead reduces 1 to 3, the sell fills 3 against 1 and 4 against 2; 2 remains with 1. Use separate starting books for these two tests.

L08 Implement replacement semantics

Prerequisite: L07. Suggested work unit: one or two focused sessions.

Understand the mechanism

Amendment is a state transition with a declared priority rule. Validate it against the current resting order before mutation. The new quantity means new remaining quantity. A same-price reduction changes only remaining size and level totals. An increase or price change becomes a priority-losing resubmission with the same order ID.

Work through an example

Orders 1 and 2 each rest at bid 100. Order 1 originally had 10 and has filled 4, so it now has 6. Amend 1 to remaining 8 at price 100: it is an increase and moves behind 2. Amend from 6 to 3 instead: it stays ahead. Request zero: reject and leave the original order unchanged.

Build it in this order

1. Write a transition table for unknown ID, bad price, zero quantity, no-op, reduction, increase, and price change. Add a test for each before implementing that path.
2. Implement no-op and same-price reduction in place. Preserve arrival priority and locator identity; adjust the aggregate with checked arithmetic.
3. For a priority-losing amendment, validate and prepare any needed output first. Remove the old active order and resubmit an internal command. Avoid incorrectly rejecting its own retained ID as a duplicate.
4. Test an amended order that crosses several levels. Record the new command sequence and, if residual rests, a new arrival priority. Maintain a ledger of explicit quantity adjustments.

Check the result

A partially filled amendment uses remaining quantity as specified. Increase loses priority, reduction preserves it, and failed validation preserves both the original state and FIFO position. An amendment that fully executes leaves no active locator.

If it fails: If the original disappears after a rejected amendment, you validated too late. If an increase keeps the queue position, you reused the reduction path. Allocation failure still follows the fatal-instance policy until a later capacity design provides preflight guarantees.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Build an oracle that can disagree

Your reference implementation is deliberately simple. Represent active orders as a vector of values. To choose a maker, scan for the best crossing price and then the smallest arrival priority. Erasing a vector element can be slow; that is acceptable in the oracle. It must not call the production matching, cancellation, or FIFO helpers.

Write the comparison boundary

For each typed command, run both books from the same preceding state. Compare success/rejection reason, command sequence, ordered trades, affected quantity, resting residual, expired residual, and a canonical order-level snapshot. Run an independent invariant checker too. A final-state-only comparison can miss a wrong intermediate trade that happens to lead to the same depth.

Validation order must be deterministic when several fields are invalid. Use this project order: command kind and enum encoding; numeric ranges; active-ID duplicate or target lookup; configured capacity limits. Adapter failures, such as a malformed text token or invalid wire record, never reach the engine and receive no engine sequence. Once a typed command is submitted to the engine, an ordinary domain rejection receives a sequence. Keep adapter validation and engine validation as separate tested layers. Pin this order in both implementations and a named test.

Generate situations, not just random numbers

Family	How to create it	Bug it can reveal
FIFO collisions	Use a narrow price range, many adds	Reversed equal-price priority
Active cancellations	Choose IDs from the reference's active set	Stale locators, wrong cleanup
Unknown and duplicate IDs	Deliberately reuse or invent targets	Validation after mutation
Sweeps	Seed several opposite levels, then cross them	Premature loop exit, wrong price
Amendment churn	Mix reductions, increases, and price changes	Incorrect priority transitions
Drain and reuse	Fully empty, refill, repeat IDs	Forgotten index entries

The generator can inspect the reference's state to choose an active ID, but it must not force the production book to agree by copying reference mutations. Record counts of each outcome. Ten thousand unknown cancellations are not evidence that active cancellation works.

Preserve the first mismatch

Save initial state, configuration, and every command through the failing command. Include both results and snapshots. A seed helps diagnosis, but standard-library random distributions may differ between implementations. The concrete command file is the regression input.

To minimize a failure, remove a chunk of commands, replay the remaining commands from a fresh initial state, and keep the deletion only if the same failure predicate persists. Renumber replay sequences contiguously if needed. Deleting setup can turn an active cancel into an unknown cancel; that is acceptable only if the original mismatch still reproduces. Then try smaller prices, quantities, and IDs one change at a time. Do not minimize into an unrelated parser failure.

Small worked failure

```
1 NEW 10 SELL 101 4
2 NEW 11 SELL 101 6
3 NEW 20 BUY 102 5
```

Expected trades: maker 10 for 4, then maker 11 for 1, both at 101. Expected state: ask 11 for 5. A production queue that inserts at the front trades with 11 first. This three-command fixture exposes the problem without a benchmark or a network.

Exercise: both models call a shared `choose_best_maker` function. You reverse its FIFO comparator and all differential tests pass. What is missing? Answer: independence. Keep common plain types, but implement maker selection separately. Add this literal fixture so even two agreeing models cannot silently redefine the required behavior.

L09 Build an independent reference and generator

Prerequisite: L08. Suggested work unit: one or two focused sessions.

Understand the mechanism

Differential testing compares two implementations of the same contract. The reference should share types and input policy where appropriate, but not the matching or queue-manipulation code whose mistakes it must expose. A slow vector of active orders can scan for the best crossing maker using price and arrival priority.

Work through an example

If the optimized book reverses FIFO at price 101 and the reference independently selects the smallest arrival priority there, their first trade IDs diverge. If both implementations call the same faulty `select_maker` helper, the comparison can pass despite the bug.

Build it in this order

1. Implement `ReferenceBook` with a vector and scans. Independently select makers, cancel, and amend. Compare exact ordered trades, rejection reasons, remaining quantities, and canonical snapshots after every command.
2. Build a deterministic random generator with a recorded seed. Mix valid active IDs with unknown and duplicate IDs. Use narrow price ranges to create FIFO collisions and crossing sweeps.
3. Run fixed families: mostly adds, mostly cancels, balanced churn, deep non-crossing books, many orders at one price, and replacement-heavy streams. Track which operation and outcome categories were exercised.
4. Save the full failing prefix at the first mismatch. Remove chunks and simplify quantities while preserving the same mismatch. Turn the minimized trace into a named regression test.

Check the result

The suite compares both implementations after each command and checks invariants independently. Introduce a temporary wrong trade price or FIFO selection and confirm a test catches it, then restore the code. Replay a saved failing input without relying on its original generator.

If it fails: A generator that cancels only unknown IDs never tests deletion. Random seeds alone may not reproduce across different standard-library distributions; saving actual commands is the reliable regression artifact.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Validate boundaries before conversion or mutation

Memory safety and domain validity are different checks. The Checked g++ build diagnoses many invalid libstdc++ iterator and container operations. It does not instrument every raw pointer access, prove that a quantity is positive, or detect every data race. Keep deterministic domain tests beside checked-container runs. [5, 6]

Parse a bounded decimal field

Avoid casting a negative signed value to an unsigned quantity. Check the entire token, including leftover characters, before using it. This small parser accepts digit-only decimal text and enforces an upper bound without overflowing.

```
// Includes: <cstdint>, <optional>, <string_view>.
std::optional<std::uint64_t> parse_positive(
    std::string_view s, std::uint64_t limit) {
    if (s.empty()) return std::nullopt;
    std::uint64_t value = 0;
    for (char c : s) {
        if (c < '0' || c > '9') return std::nullopt;
        const auto digit = std::uint64_t(c - '0');
        if (digit > limit || value > (limit - digit) / 10)
            return std::nullopt;
        value = value * 10 + digit;
    }
    if (value == 0) return std::nullopt;
    return value;
}
```

The condition proves $\text{value} * 10 + \text{digit} \leq \text{limit}$ before evaluating that expression. This parser accepts leading zeros but rejects a sign, embedded whitespace, and a suffix. State that grammar in replay tests. IDs use `UINT64_MAX`; prices and quantities use the contract's smaller bounds.

Guard aggregates too

To add q to total, check $q \leq \text{max} - \text{total}$ first. To remove q , establish $q \leq \text{total}$ first. A test that constructs total near `UINT64_MAX` is valid for the arithmetic helper even if the normal engine workload would need an impractical number of orders to reach it. Test the helper's full range, then test configured engine limits with small capacities.

Use a before snapshot for every rejected new, cancel, and amendment case. Compare trade output as well as state. For the deliberate Checked smoke, use a separate executable containing `std::vector<int> v(1);` followed by `v[1] = 7;`. Compile it with `-D_GLIBCXX_DEBUG` and require a debug-container diagnostic. Keep it outside the passing test suite. Do not run the unchecked version: that access would have undefined behavior. This verifies container checking, not general memory instrumentation.

Exercise: total is `UINT64_MAX - 2` and q is 3. Answer: reject before addition. For text `'-1'`, reject before conversion. For a same-price amendment to zero, reject before touching the node. Neither a fast run nor a Checked-build pass replaces these checks.

L10 Test failure paths with g++ on Windows

Prerequisite: L09. Read the preceding L10 teaching pages and use the Checked configuration from L02.

Understand the mechanism

Checked libstdc++ containers diagnose many invalid iterator and container operations. They do not instrument all memory accesses or detect every race. Explicit validation, bounds checks, and invariants remain required.

Work through an example

For `uint64_t` total, check `q <= max - total` before addition. Reject textual -1 before unsigned conversion. A zero-quantity amendment leaves both the original order and its FIFO position unchanged.

Build it in this order

1. Run Debug, Checked, and Release tests. Keep invariant validation callable when `NDEBUG` is defined. Record the actual toolchain and configurations.
2. Test malformed enum values, negative/overlong text, zero, maximum accepted quantity, and aggregate-limit helpers. Compare full before/after state on rejection.
3. In a separate smoke executable, use `vector<int> v(1); v[1] = 7;` compiled with `_GLIBCXX_DEBUG`. Require a container diagnostic. Keep this intentional failure outside the passing suite.
4. Exercise repeated fill, drain, cancel, and ID reuse. Check destruction order. Preserve the fatal-instance allocation-failure policy until a bounded implementation supplies preflight guarantees.

Check the result

The suite passes, the Checked smoke diagnoses the bad vector access, and boundary tests fail when validation is removed. Describe the result as g++ Checked-container testing; do not label it ASan-, UBSan-, or TSan-tested.

If it fails: verify the Checked definitions reach every linked target. Do not mix prebuilt unchecked container types with Checked code. Missing raw-pointer diagnostics are a coverage limit; retain explicit bounds and lifetime reasoning.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

A replay file is an executable specification

Determinism means the same initial state, configuration, and ordered commands produce the same logical results and final state. A seed, timestamp, or checksum alone does not establish that property. Keep the actual commands and compare canonical values.

Freeze a small text grammar

ASCII tokens are words/numbers encoded with the ASCII character set. LF is a line-feed byte; CRLF is a carriage-return byte followed by line feed. Use tokens separated by spaces; accept either line ending; reject extra tokens. Ignore blank lines. The first nonblank line is LOB-TEXT 1. Subsequent sequence numbers start at 1 for an empty session and increase by one. For snapshot recovery, the file declares the restored starting sequence instead. Side is BUY or SELL; numeric fields follow L10's parser.

```
LOB-TEXT 1
1 NEW 10 SELL 101 4
2 MARKET 20 BUY 3
3 AMEND 10 102 2
4 CANCEL 10
```

NEW fields are sequence, NEW, ID, side, price, quantity. MARKET omits price. AMEND fields are sequence, AMEND, ID, new price, new remaining quantity. CANCEL ends after ID. Command syntax errors stop file loading with a line number. Domain-valid syntax that requests an unknown ID becomes a normal engine rejection.

Hand-derived output for that file

Command 1 rests ask 10 for 4 at 101. Command 2 trades maker 10/taker 20 for 3 at 101, leaving ask 10 for 1. Command 3 changes the price to 102 and remaining quantity to 2, loses priority, and rests there. Command 4 removes 2 with no trade. Final state is empty; the next command sequence is 5.

Represent outputs in a declared field order. Print each trade in execution order and each snapshot in the canonical side/price/FIFO order. Do not serialize padding bytes, pointer values, or hash-table iteration order. If you add a digest, define the encoded bytes first and keep full-value comparison for tests.

Make replay available to every later tool

The runner accepts a typed vector and an initial state; the CLI parser is an adapter that produces that vector. Tests can call the runner without opening a file. Benchmarks can load once and time only command execution. Networking can decode a frame into the same command type. None of these adapters needs a second matcher.

Exercise: omit sequence 3 and send CANCEL as sequence 4. Answer: reject the file's sequence gap before treating it as a complete valid replay. If a valid sequence 3 CANCEL names ID 999, process it as UnknownOrder and advance the engine sequence. These are different failure layers and should have different tests.

L11 Replay commands and preserve evidence

Prerequisite: L10. Suggested work unit: one or two focused sessions.

Understand the mechanism

Replay feeds an ordered command sequence into a fresh engine and records the results. It is already useful before binary serialization. Commands express intent; trades and book changes are outputs. Keep those directions separate. A deterministic result also requires a defined output order, not `unordered_map` iteration order.

Work through an example

Use lines such as 1 NEW 10 SELL 101 4 and 2 NEW 20 BUY 102 3. The second command produces maker 10, taker 20, quantity 3, price 101; the remaining ask is 10 with quantity 1. Replaying twice from empty state must produce the same result sequence and snapshot.

Build it in this order

1. Define a typed command vector and a readable line format with an explicit format version. Start with New, Market, Cancel, and Amend; write one example of each in `docs/replay.md`.
2. Implement a runner that takes an initial state and command vector. Keep text parsing separate and report line number plus reason for malformed input. Require contiguous positive command sequences in each file.
3. Sort snapshots by declared side, price, and FIFO priority. Compare canonical field values, not raw object bytes. Save the commands and expected outputs for the minimized failures from L09.
4. Add a digest only as a convenience. Hash an explicit canonical byte or text representation with a documented algorithm; compare full outputs in correctness tests and do not rely on `std::hash` as a portable file checksum.

Check the result

The same file produces identical trades, rejections, and final state across repeated runs. A missing or duplicate sequence is rejected. CRLF and LF line endings parse as intended. File read time and matching time can be measured independently.

If it fails: If output changes, look for unordered iteration, timestamps, uninitialized fields, a nonempty starting book, or input-dependent validation order. Never feed output executions back into the matching-command stream.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Measure a defined piece of work

A performance claim needs a unit, a workload, and a timing boundary. Start with commands per second and average nanoseconds per command for a pre-generated batch. The matching core creates its normal results during timing; generation, initial-state construction, parsing, and final printing stay outside.

Construct a repeatable experiment

Choose a deterministic file with declared ratios of adds, cancels, amendments, markets, and rejected commands. Record the starting live-order count, price-level count, and ending count. Replay it on the reference once for correctness. Warm a disposable engine, discard it, and construct a new measured engine from the same starting state for every repetition.

```
// Harness sketch: adapt names to your public API.
auto commands = load_commands(workload_path); // outside timing
warm_disposable_engine(initial_state, commands);
for (int repetition = 0; repetition < 7; ++repetition) {
    auto engine = make_engine(initial_state);
    std::uint64_t trade_count = 0;
    const auto begin = std::chrono::steady_clock::now();
    for (const auto& command : commands) {
        auto result = engine->process(command);
        trade_count += result.trades.size();
    }
    const auto end = std::chrono::steady_clock::now();
    record(end - begin, commands.size(), trade_count);
}
```

Store and print the recorded counts after timing so outputs remain observable. Validate final snapshots outside timing. Use runtime-loaded input and inspect suspiciously tiny timings; a compiler may eliminate work whose results have no observable use. If using Google Benchmark, its `DoNotOptimize` and `ClobberMemory` help express measurement barriers, but they do not repair a wrong workload boundary. [11]

Compute the units

If $N = 100,000$ and $\text{elapsed} = 0.040$ seconds, throughput is $N/\text{elapsed} = 2,500,000$ commands/s. Average batch cost is $\text{elapsed} * 1e9/N = 400$ ns/command. That average includes any harness work kept inside the boundary. Label it as measured command-processing cost rather than pretending you isolated one CPU instruction.

Use Release, without a debugger or sanitizer. Record compiler, flags, commit, workload identity, CPU, Windows version, power mode, and whether affinity was set. On your hybrid CPU, core placement can change results. Alternate baseline and candidate runs, retain all repetitions, and report a median plus spread rather than selecting the fastest observation. Profile separately from headline timing.

Percentiles need individual observations

For an operation-latency experiment, preallocate storage and record a start/end interval around each sampled operation. Sort n observed durations. A declared nearest-rank p -quantile is element $\text{ceil}(p*n)-1$ in zero-based indexing. With samples [1, 2, 2, 3, 20] microseconds, p_{50} is 2 and p_{99} is 20. Five samples cannot support a useful tail-latency claim.

Measure timer overhead and the effect of recording samples using the same harness. Do not subtract one constant from each sample and claim precise corrected tails. Compare sampled and unsampled throughput to quantify perturbation. Windows high-resolution timing is an interval clock facility; a displayed nanosecond unit is not evidence of nanosecond measurement accuracy. [12]

Google Benchmark's statistics across repeated runs are not percentiles of individual command latencies. One million individual samples place roughly a thousand observations above $p_{99.9}$, but independence, workload duration, and noise still matter. Keep those raw observations if you report that percentile. [11]

Include waiting when waiting is the question

Later, end-to-end latency starts at a declared client send/scheduled-arrival boundary and ends at a response boundary on a compatible clock. If a load generator waits for each response before sending the next request, it reduces offered load when the server slows. Add a fixed-rate arrival experiment to study overload and record queueing plus service time. Never compare that result to an in-process batch average as if they measure the same thing.

Exercise: run a cancel-heavy batch twice on the same engine. The second run is faster. Answer: it may be rejecting now-missing IDs; you changed the workload. Rebuild the initial state for each run. Complete the first-release review after the practice page; extension branches remain optional.

L12 Establish a reproducible benchmark

Prerequisite: L11. Suggested work unit: one or two focused sessions.

Understand the mechanism

Throughput measures completed commands per second; service time measures work inside a chosen operation boundary. End-to-end response latency may also include queueing, transport, and publication. Start with batch throughput and average service cost. Individual-event percentiles require separate sampling; Google Benchmark repetition statistics are not those percentiles. [11]

Work through an example

If a measured batch takes 40 ms for 100,000 commands, its average cost is 400 ns per command and throughput is 2.5 million commands per second. Neither number tells you p99. A second run that starts with all the cancellation targets already deleted measures a different workload.

Build it in this order

1. Pre-generate commands with fixed occupancy, add/cancel/amend ratios, and crossing behavior. Prebuild the same initial book for each candidate. Warm up on a disposable engine, then recreate the measured initial state.
2. Time a large batch using `steady_clock`; exclude generation, reset, I/O, and final formatting. Consume results so work remains observable. Include normal engine output creation unless the measurement explicitly defines a narrower scope.
3. Run several repetitions in Release without a debugger or sanitizer. Record compiler, flags, commit, CPU, Windows version, power mode, workload file, and live-order range. On a hybrid CPU, scheduling can move work between different core types.
4. For a separate latency experiment, preallocate sample storage, measure timer overhead and sampling perturbation, and report count, boundary, and resolution. Do not call batch-average percentiles order latency. Include queued time when measuring the future network service.

Check the result

A second person can reproduce the workload and measurement procedure. Store raw repetitions, not just the fastest run. If p99.9 has too few observations or timer noise dominates, defer that claim. At one million samples, only about one thousand lie above the 99.9th percentile.

If it fails: Changing book occupancy during warm-up, timing Debug, or including RNG often dominates the result. A request-response load generator can miss overload by sending less work when the server slows; later queue tests need controlled arrival rates.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

First release completion review

After L12, package the matching core as a finished release. A project with clear semantics, independent tests, reproducible inputs, and a measured baseline is a valid deliverable. Extensions can proceed while you apply for roles.

- ☐ Matching and amendment policies are documented and match tests.
- ☐ A saved command file reproduces the same results from an empty book.
- ☐ Scenario, invariant, differential, and Checked runs have recorded evidence.
- ☐ The baseline states its workload, build, machine, and measurement scope.
- ☐ A fresh checkout can configure, build, and run the tests on Windows.
- ☐ I can explain a partial fill, a failed amendment, and locator lifetime without code assistance.

Write a short reproducible demonstration

1. Record the commit and build commands. Show the hand-worked sweep, a priority-preserving reduction, and a priority-losing amendment.
2. Run the replay fixture and tests. Show one previously minimized bug and its regression test.
3. Show the baseline results with units and scope. Explain one limitation and select a single next experiment.

Choose the next branch

Goal	Next lesson	Finish when
Understand allocation	P01	You can account for resource calls and explain the measured effect.
Improve storage layout	P02 then P03	A second representation passes the same contract and its trade-off is documented.
Build a networked simulator	E01	Messages have a specified encoding and exact decode tests.
Learn concurrency	C01	A bounded queue drains and shuts down under controlled tests.

Release tag and commit

Demo command or test

Next selected lesson

Describe only implemented work on your resume. A suitable structure is: implemented [features]; validated with [specific tests]; measured [defined workload and metric]. Fill the brackets from your evidence. Avoid treating any version number as a guarantee of recruiting value.

Count allocations before changing them

Terms introduced here

An allocator supplies raw storage for objects. A memory resource is the interchangeable storage provider used by C++ polymorphic memory resource (PMR) containers. A pool groups reusable blocks; upstream names the provider from which it obtains those blocks. Alignment means an address must be a multiple of a required boundary. Churn means repeated allocation and removal.

Allocation has two levels in a pool experiment. A container requests storage for nodes. A pool obtains larger chunks from an upstream resource and reuses blocks within those chunks. Counting only upstream requests can hide thousands of logical container allocations.

Understand PMR through one container

`new_delete_resource` is the standard provider that allocates and releases storage through ordinary `new/delete`. `std::pmr::memory_resource` is a runtime interface with `allocate`, `deallocate`, and `equality` operations. A `pmr` container forwards storage requests to the resource it was given.

`unsynchronized_pool_resource` maintains reusable pools without promising concurrent access. Use it on one owning thread. A `monotonic_resource` instead retains storage until release or destruction; individual `free`s do not give it the same reuse behavior. [13]

```
// Includes: <memory_resource>, <list>.
std::pmr::unsynchronized_pool_resource pool;
std::pmr::list<int> values{&pool};
for (int i = 0; i < 1000; ++i) values.push_back(i);
values.clear();
// values is destroyed before pool because locals die in reverse order.
```

Do not call `pool.release()` while `values` or another dependent container still owns nodes. The resource owns storage; the container manages object lifetimes in that storage. In a class, declare the resource before its dependent containers so destruction reverses that order. Explicitly initialize each nested PMR container with the intended resource.

Build a transparent counting wrapper

A virtual function allows a base interface to dispatch to a derived implementation; override checks that your function replaces that interface method. noexcept promises not to throw. The three overrides have these signatures:

```
void* do_allocate(std::size_t bytes, std::size_t alignment) override;
void do_deallocate(void* p, std::size_t bytes,
                  std::size_t alignment) override;
bool do_is_equal(const std::pmr::memory_resource& other)
    const noexcept override;
```

Subclass memory_resource. In do_allocate(bytes, alignment), first ask the upstream resource to allocate, then increment successful-call count and live bytes. In do_deallocate(p, bytes, alignment), forward the same pointer, size, and alignment and decrease live bytes. Track peak live bytes. Implement do_is_equal by identity for this simple wrapper. Do not allocate a logging string from inside the allocation hook; that can recursively allocate.

Measure two positions separately: logical counter between container and pool, upstream counter between pool and new_delete_resource. Keep both counters alive until the pool and container finish. A minimal declaration order is upstream counter, pool, logical counter, container; destruction then proceeds container, logical counter, pool, upstream counter.

Interpret one fill/drain experiment

After clearing 1,000 list nodes, the logical counter should report zero live requested node bytes. The upstream counter may still report retained pool storage. Refill the same list. If retained chunks are reused, upstream requests can fall while logical node requests remain. Report both, plus retained capacity and elapsed time. A pool can be slower on a workload with little reuse.

Exercise: reserving the ID hash table removes rehashing but allocation counts remain high. Answer: reserve controls bucket capacity; hash nodes and order nodes can still allocate. Measure the individual containers before deciding where pooling helps. Keep or revert one change after identical replay and benchmark runs.

P01 Measure allocation and try a pool

Prerequisite: L12. Suggested work unit: one or two focused sessions.

Understand the mechanism

A node-based container asks an allocator for storage when it grows. A pool can satisfy repeated small allocations from reusable blocks, but it also retains memory. Count the requests made by the container separately from the calls the pool makes upstream. Reserving `unordered_map` buckets does not eliminate allocation of each element node.

Work through an example

Place one counting `memory_resource` between a PMR list and a pool, and another between the pool and `new_delete_resource`. Inserting and deleting 1,000 nodes may produce many logical node requests but fewer upstream allocations. Report both counts and peak live bytes; do not label a pooled system zero-allocation because only upstream calls were measured.

Build it in this order

1. Write a small `memory_resource` wrapper that increments allocation/deallocation counts and byte totals, then forwards the original size and alignment. Exercise it with `pmr::list<int>` before touching the engine.
2. Run a fixed fill/drain workload with the default resource. Add reserve for a known hash capacity as a separate change and measure it separately.
3. Switch one selected container to `unsynchronized_pool_resource`. Keep the resource alive longer than its containers; explicitly pass it to nested PMR containers rather than assuming propagation.
4. Rerun differential tests and identical benchmarks. Record retained bytes, upstream calls, logical requests, and elapsed time. Keep or revert the change based on that evidence.

Check the result

You can explain where every counted allocation originates and what the pool retains after a drain. The engine emits identical outputs. A resource can be destroyed only after its dependent containers. A negative performance result still completes the experiment.

If it fails: A monotonic resource does not recycle individual frees in the same way and can grow during churn. If memory remains allocated after cancel-all, distinguish retained reusable capacity from leaked unreachable storage.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

An indexed pool owns slots and links

Terms introduced here

A slot is one position in a fixed storage array. An index is its numeric position. Links are indices of neighboring live orders. A free list records unused slots. A generation is a version counter that distinguishes successive occupants of one slot. A sentinel is a reserved value meaning no valid position.

Replace the per-level list only after the locator rules are understood. A fixed vector of slots owns all order storage. Each live slot contains an order, previous and next indices, generation, and an owning-level identity. Each price level stores head and tail. Physical slot order does not define FIFO.

Define empty and live states

Use a sentinel index equal to capacity, outside $[0, \text{capacity})$. A free list contains every unused slot exactly once. Each live slot belongs to exactly one level and is reachable from its head. No free slot is reachable from a level. A handle is (index, generation); validate index range, live flag, and matching generation before returning access.

Acquire removes one free index and marks it live. Release clears ownership, increments its generation, and returns it to the free list. If increment would wrap, retire the slot for the remainder of the instance or reject further reuse. Otherwise an old external handle could eventually become valid again.

Unlink all four cases with one rule

```
p = slots[i].prev; n = slots[i].next
if p exists: slots[p].next = n
else:       level.head = n
if n exists: slots[n].prev = p
else:       level.tail = p
```

This handles head, middle, tail, and sole-node removal. For [4, 9, 2], unlinking 9 connects 4 to 2 in both directions. Appending i sets prev to the old tail and next to the sentinel, updates the old tail or empty head, then sets the new tail. Check size and total accounting separately.

Capacity is part of behavior

Before mutation, conservatively reserve one slot for any NewLimit that might rest. A Market does not require a resting slot, but it still needs bounded output space. An amendment losing priority can reuse its old slot after removing it; preflight its other requirements first. State whether a full book may reject a crossing limit that would in fact fill completely. This workbook permits that conservative rejection in the bounded variant when declared in the reference too.

Output capacity is independent of order capacity. A sweep can produce many trades and feed effects. For a conservative bound, count eligible makers without mutation or allocate a result buffer large enough for the declared maximum. A capacity failure must occur before the first fill or old-order removal. Do not reject halfway through an accepted sweep.

Structural checks catch different errors

Walk forward and backward from every level. Verify reciprocal links, correct head/tail sentinels, no cycles, matching level ownership, and exactly-once reachability. Verify live count plus free count plus retired count equals capacity. Then run the normal differential suite. A pool can pass trade tests while leaking slots until it later reports false capacity exhaustion.

Exercise: release slot 9 at generation 4 and reuse it at generation 5. Answer: handle (9,4) must fail and (9,5) may succeed. Clearing only the ID index is insufficient if another component retains the old handle. Contiguous allocation also does not guarantee adjacent FIFO slots after repeated cancellations.

P02 Build indexed order storage

Prerequisite: L12 and stable-handle understanding from L07. Suggested work unit: one or two focused sessions.

Understand the mechanism

An indexed pool stores order records in a fixed vector of slots. Each live slot contains prev and next indices; each level stores head and tail. An ID locates a slot, and that slot identifies its owning level. The vector owns storage, so links survive relocation only if they are indices and all assumptions are respected. A reused slot must not make an old handle valid again.

Work through an example

A level links slots 4, 9, and 2 in FIFO order. Cancel slot 9: set slot 4 next to 2 and slot 2 prev to 4, then return 9 to the free list. The physical order [2, 4, 9] is irrelevant to FIFO. A generation number lets an external handle distinguish the old occupant of slot 9 from a new one.

Build it in this order

1. Build a fixed-capacity pool exercise with acquire, release, and generation checks. Define an invalid index outside the valid range and test capacity zero and one.
2. Implement append and unlink for head, middle, tail, and the sole node. Store side and level location in the locator or slot; an ID-to-slot mapping alone cannot identify the owning FIFO.
3. Substitute this queue for list while retaining the price map. Validate forward/backward links, absence of cycles, free/live separation, and exactly-once reachability of every live slot.
4. Preflight capacity and output space before mutating a command. For this bounded variant, a conservative CapacityExceeded rejection is allowed before matching when a required free slot is unavailable. Apply that declared policy to the reference too.

Check the result

Fill to capacity, reject one additional required allocation without mutation, cancel, reuse, and drain. Stale generation handles are rejected. On workloads below capacity, outputs match the prior implementation. Capacity tests compare against a reference configured with the same policy.

If it fails: A contiguous slot array does not ensure adjacent FIFO nodes after churn. Avoid claiming locality benefits from the allocation layout alone. A generation counter must have an explicit wrap policy before it can repeat an externally visible handle value.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Measure layout and profile g++ on Windows

Terms introduced here

A price ladder is an array indexed by price within a bounded interval. Dense means many possible prices are occupied; sparse means many are empty. Profiling instrumentation adds measurement code. DWARF and PDB are different debug-information formats; an unresolved frame is a call-stack entry the viewer cannot map to a source function.

A tree stores occupied prices. A dense ladder stores every price in a bounded interval and maps tick p to $p - \text{minimum}$. A direct price lookup does not solve the separate problem of finding the next nonempty price. Model both costs before replacing the tree.

Compare storage on a common domain

For $[90,110]$, allocate 21 levels; tick 101 maps to index 11. Check the range before subtraction or conversion to `size_t`. Keep current best bid and ask indices. When the best level empties, scan toward the next occupied index, or maintain a separate occupancy structure such as an ordered set of nonempty indices. Account for its update cost and memory too.

Workload	What it exposes
Narrow, mostly occupied range	Direct addressing and compact level metadata
Wide range with two occupied prices	Memory reservation and empty-price scans
Repeated cancel of best level	Cost of finding the next best price
Heavy same-level churn	Order storage and ID-index costs

An index of 999,999,999 is a memory-design problem even if the arithmetic is constant time. Compare accepted commands under the same configured domain. A speedup from rejecting prices the baseline accepts is not an equivalent implementation.

Profile in a way that matches g++ symbols

Windows Performance Recorder (WPR) records operating-system events. Windows Performance Analyzer (WPA) displays those recordings. Use Release builds with g++ debug information for inspection; keep ordinary Release measurements separately. Windows Performance Recorder can capture CPU sampling and Windows Performance Analyzer can show process/thread CPU use, scheduling, and module activity. Install Windows Performance Toolkit if needed. WPR may need an elevated terminal to start a recording. [15, 16]

```
wpr -profiles
wpr -start CPU -filemode
.\build\profile\lob_bench.exe
wpr -stop profile.etl
```

Create build/profile using Ninja and g++ with CMAKE_BUILD_TYPE=RelWithDebInfo. Add a lob_bench target in L12 before running this example. In WPA, open profile.etl, add CPU Usage (Sampled), filter to your process and the measured interval, then inspect thread/module totals. g++ emits DWARF information; do not assume a Windows PDB-oriented stack viewer resolves every GCC function name. Module or address-only frames are a limitation to record, not a reason to switch compilers. [15, 16]

For a function-level starting point that stays with g++, make a separate GNU profiling experiment using -pg during both compilation and linking and gprof from the same toolchain if supplied. Confirm a tiny known function produces a readable profile before profiling the engine. If that smoke fails, use coarse scoped timers outside headline runs and GDB source inspection. Neither -pg nor extra timers belongs in performance comparisons.

Turn an observation into one experiment

If most measured time goes to ID lookup, replacing price storage may have little effect. If repeated empty-ladder scans dominate sparse input, track occupied indices as a separate candidate. Sampling a function often means time was spent there or below it; it does not establish cache-miss counts. Report hardware counters only when an actual tool collected them.

Exercise: a ladder wins on a 21-tick domain but loses on 10,000 sparse ticks. Answer: keep both results and explain the domain trade-off. Complete the experiment even if you retain the tree. A useful result is a reproducible decision with a measured reason.

P03 Compare price layouts and profile on Windows

Prerequisite: L12, optionally P01/P02. Read the preceding P03 teaching, including its g++ profiling limitations.

Understand the mechanism

A dense price ladder trades a bounded domain and memory for direct lookup. Finding the next occupied price remains a separate cost. A tree handles sparse occupied prices without allocating every possible tick.

Work through an example

Ticks 90 through 110 need 21 slots; 101 maps to 11. A domain spanning 1 through 1,000,000,000 makes a full ladder impractical. Direct access does not make a billion-slot scan cheap.

Build it in this order

1. Run a Release baseline long enough to inspect. Use the included WPR/WPA procedure for CPU and scheduling data; confirm symbol resolution before making function-level claims about g++ output.
2. Record the measured hot work before choosing a change. Implement a bounded ladder as a separate variant and reject prices outside its declared domain.
3. Test dense, sparse, and repeated-best-removal workloads. Compare only identical accepted commands and initial states.
4. Report elapsed time, memory capacity, allocation counts, and profile limitations. Use a separate g++ -pg/gprof experiment only if its smoke test works; keep profiling instrumentation out of headline timing.

Check the result

The candidate matches the reference on its domain. You can explain next-best search and memory cost. Record a keep/revert decision using more than one workload.

If it fails: a faster run does not establish fewer cache misses. Address-only Windows stacks do not prove a particular C++ function is hot. Use the documented fallback measurements and state their scope.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Bytes have an explicit layout

Terms introduced here

Serialization converts typed fields into a defined byte sequence; decoding reverses it. A binary format uses byte values rather than readable decimal tokens. Wire means the transmitted representation. A protocol defines messages and how to interpret them. A record contains a header describing it and a payload carrying its fields. An offset counts bytes from a stated start; big-endian places the most significant byte first. u8, u16, u32, and u64 mean unsigned 8-, 16-, 32-, and 64-bit fields.

The custom protocol is small enough to define completely here. Every multibyte integer is unsigned big-endian. Widths are fixed, with no padding. A u64 occupies eight bytes even if a C++ struct would align surrounding fields differently. Prices are validated as positive wire integers before conversion to signed internal Price.

Header and payload offsets

Header offset	Field	Bytes	Rule
0	Version	1	Exactly 1
1	Type	1	1 NewLimit, 2 Market, 3 Cancel, 4 Amend
2	Payload bytes	2	Excludes header; maximum 4096
4	Sequence	8	Positive; namespace set by adapter

Type	Payload fields, offsets relative to payload	Payload / total
1	ID u64@0; side u8@8; price u64@9; quantity u64@17	25 / 37
2	ID u64@0; side u8@8; quantity u64@9	17 / 29
3	ID u64@0	8 / 20
4	ID u64@0; price u64@8; remaining u64@16	24 / 36

Side 0 is Buy and 1 is Sell. ID and numeric ranges follow the matching contract. Unknown types, other side bytes, wrong exact lengths, sequence zero, and unsupported versions are rejected. This is a custom format; OUCH is only a reference for the distinction between order entry and market data. [17]

Encode by shifting, not casting

```
// Includes: <cstdint>, <cstdint>, <vector>.
void append_u64(std::vector<std::byte>& out, std::uint64_t x) {
    for (int shift = 56; shift >= 0; shift -= 8)
        out.push_back(std::byte((x >> shift) & 0xffu));
}
```

The first byte contains the most significant eight bits. Encode a u16 with shifts 8 and 0. Do not write `reinterpret_cast<char*>(&object)` to disk: object padding, alignment, host byte order, and enum representation are not this format.

Exercise: Cancel ID 0x0102030405060708, sequence 7. Expected bytes: header 01 03 00 08 00 00 00 00 00 00 00 07; payload 01 02 03 04 05 06 07 08. Total 20. Use that exact golden fixture as well as round trips. An encoder and decoder can agree on the same incorrect byte order.

E01 Specify a portable binary format

Prerequisite: L11 and the first release. Suggested work unit: one or two focused sessions.

Understand the mechanism

Serialization maps logical fields to bytes in a declared order. C++ object memory also contains alignment and representation details, so dumping a struct is not a file format. Begin with a tiny format you own. Use the same command payload definitions for replay and later transport, while documenting each stream's sequence namespace.

Work through an example

Encode Cancel with ID 0x0102030405060708 and sequence 7. The 12-byte header is version 1, type 3, payload length 8, then sequence 7, all multibyte fields big-endian. The eight ID bytes are 01 02 03 04 05 06 07 08. Total record length is 20.

```
01 03 00 08 00 00 00 00 00 00 00 07
01 02 03 04 05 06 07 08
```

Build it in this order

1. Write docs/wire.md before the encoder. Define a 12-byte header: u8 version, u8 type, u16 payload length, u64 sequence. Bound payload length at 4096 and support only version 1.
2. Specify type 1 NewLimit payload as u64 ID, u8 side, u64 positive price ticks, u64 quantity, totaling 25 bytes. Type 2 Market is ID, side, quantity, totaling 17. Type 3 Cancel is ID, totaling 8. Type 4 Amend is ID, price, remaining quantity, totaling 24.
3. Define side 0 as Buy and 1 as Sell. Decode wire prices into a checked unsigned temporary, validate the contract range, then convert to the internal signed Price. Sequence 0 is invalid.
4. Write byte-append helpers and an encoder. Add golden byte tests for each command and boundary value before writing any sockets.

Check the result

The Cancel example yields the exact bytes shown in the example. Payload lengths exclude the header. NewLimit, Market, Cancel, and Amend records have lengths 37, 29, 20, and 36 bytes. The documentation states what sequence means in replay versus a client session.

If it fails: A round-trip test can miss an encoder and decoder that share the same wrong byte order. Maintain hand-derived golden bytes. Do not reinterpret_cast a byte buffer into a message struct.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Decode without reading outside the buffer

Terms introduced here

A buffer is owned storage for bytes. An accumulator retains received bytes until complete records exist. A prefix is the beginning of a sequence; a suffix is its ending. Truncation means a record ends before its promised length. Framing is the rule that identifies record boundaries in a byte stream. Golden bytes are a hand-derived exact expected encoding.

A decoder returns Complete, NeedMoreData, or Invalid. A complete result also supplies one typed message and its consumed byte count. NeedMoreData consumes zero. It never mutates the book. File EOF converts a leftover incomplete prefix into TruncatedRecord; an active TCP connection can keep waiting for the missing bytes.

Bounds-check each integer read

```
// Includes: <cstdint>, <cstdint>, <optional>, <span>.
std::optional<std::uint64_t> read_u64(
    std::span<const std::byte> bytes, std::size_t& offset) {
    if (offset > bytes.size() || bytes.size() - offset < 8)
        return std::nullopt;
    std::uint64_t value = 0;
    for (int i = 0; i < 8; ++i)
        value = (value << 8) |
            std::to_integer<unsigned char>(bytes[offset++]);
    return value;
}
```

Subtraction after checking offset avoids unchecked offset + 8 overflow. Since exactly eight bytes enter a u64, each left shift builds the intended representation. Implement a separate u16 reader or a small width-checked generic helper; do not cast an unaligned span to a u64 pointer.

Parse in layers

Wait for the 12-byte header. Decode locals, validate version/type and the exact expected payload size. If the declared size is impossible for that type, return Invalid now. Otherwise wait for the complete record. Decode all fields into locals, validate ranges and enum values, then construct the typed command. A malformed header must not force the accumulator to reserve arbitrary memory.

In a stream accumulator, append a bounded received chunk. While a complete frame exists, dispatch it and advance a read offset. Preserve the incomplete suffix. Compact only when useful; repeatedly erasing the first byte from a vector can cause unnecessary copying. Cap buffered bytes, frames per dispatch visit, and outbound bytes.

Test fragmentation and coalescing exhaustively

For each golden message of length L , split after byte k for every k in $[0, L]$. Feed both parts into a fresh accumulator and require exactly one decoded message. For $k < L$, the first part must not dispatch a message. Concatenate three records and require three commands in order. Add a fourth partial prefix and ensure it remains buffered.

Mutate type, version, side, quantity, and length. Test empty input and each truncation. Bounded arbitrary byte vectors should always terminate with one of the documented outcomes. Run these under Checked and use explicit span bounds: a libstdc++ diagnostic is not a substitute for safe raw-byte parsing.

Exercise: a Cancel header claims nine payload bytes and only eight arrive. Answer: Invalid as soon as the full header is available, because Cancel requires exactly eight. A correct header followed by three payload bytes is NeedMoreData, or TruncatedRecord at file EOF.

E02 Decode bytes and reject malformed records

Prerequisite: E01. Suggested work unit: one or two focused sessions.

Understand the mechanism

A decoder needs to distinguish incomplete input from an invalid complete record. TCP may provide only part of a valid record; a completed file ending halfway through one is truncated. Decode fields into local variables and validate lengths before accessing payload bytes or constructing a domain command.

Work through an example

The Cancel header promises eight payload bytes. Receiving only three of those bytes means `NeedMoreData` in a stream accumulator. Reaching file EOF at that point means `TruncatedRecord`. A Cancel header promising nine bytes is malformed even if all nine are available.

Build it in this order

1. Define `DecodeResult` as a status, optional command, consumed byte count, and error reason. A successful call consumes one record; `NeedMoreData` consumes none. Reject unknown types and unsupported versions.
2. Implement bounds-checked `u16/u64` reads on `span<const byte>`. Check available size by subtraction rather than unchecked `offset + length` arithmetic. Validate exact payload length for each type.
3. Split each golden record at every byte boundary and feed the chunks through an accumulator. Concatenate three records and extract all three without losing the remaining prefix.
4. Generate bounded arbitrary byte vectors and mutate golden lengths, versions, enums, IDs, and quantities. Run decoder tests in `Checked`. Add a CLI file converter only after isolated decoding works.

Check the result

Every truncation returns a defined result, complete malformed records are rejected, and decoding an encoded valid command reproduces its fields. The decoder does not mutate the book. Stream buffering never grows beyond the declared frame limit plus a documented batch allowance.

If it fails: Treating incomplete as invalid breaks fragmented TCP. Treating invalid length as incomplete lets a peer force unbounded waiting or buffering. Reject the header as soon as its type, version, or size cannot be valid.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Specify responses and visible book effects

Terms introduced here

A client submits commands; a server processes them. A subscriber receives published market data, an output stream describing visible changes. An event is one such change or report. A bundle groups one command's results. Correlation identifies which request a response belongs to. A token is a client-chosen order identifier. Preflight here counts eligible makers and checks output capacity before any mutation.

The client needs a result correlated with its request. The subscriber needs an ordered account of visible state changes. These are different records. Define the custom encoding below to complete the earlier command format; it is intentionally independent of OUCH and ITCH. [17, 18]

Result bundle on the client connection

Reuse the 12-byte header; use type 0x80 and put the client request sequence in its header. The payload fields below appear in order. All integers are big-endian. This single bundle represents accepted, rejected, canceled, amended, executed, and expired outcomes without requiring several independent response formats.

Field	Width	Meaning
Engine command sequence	u64	Positive dispatch sequence
Command kind, status, reason	u8 each	Kind 1..4; status 0 success/1 reject
Client token	u64	Request's order token
Affected remaining	u64	Cancel removed size; Amend old remaining; 0 for New/Market
Resting, expired	u64 each	Residuals for this command; Cancel has both 0
Trade count	u16	Number of following trade records
Each trade	4 x u64	Maker engine ID, taker engine ID, price, quantity

The fixed payload is 45 bytes plus 32 per trade. Reason codes: 0 None, 1 InvalidKindOrSide, 2 OutOfRange, 3 DuplicateActiveId, 4 UnknownOrder, 5 CapacityExceeded. A rejection has zero trades and zero residual/affected fields. The gateway can translate a known owned taker token separately; trade IDs remain engine IDs. A protocol error disconnects before producing a matching result.

For this small network exercise, cap a command at 64 maker fills using conservative preflight before mutation, and include that configured bound in its reference. Maximum result payload is 2093 bytes, below 4096. This network cap does not retroactively change the unbounded core. A production-sized variant would need an explicit segmented result format.

Feed effect schema

Use the same header with feed-event sequence. Each event has this payload and exact length. ID means engine ID; priority is the order's FIFO priority counter.

Type	Fields in payload order	Bytes
0x20 BookAdd	ID u64, side u8, price u64, remaining u64, priority u64	33
0x21 BookExecution	ID u64, executed u64	16
0x22 BookCancel	ID u64, removed u64	16
0x23 BookDelete	ID u64	8
0x24 Trade	maker u64, taker u64, price u64, quantity u64	32

BookAdd appends only a resting residual. Execution and Cancel subtract a positive amount no larger than remaining and remove the order at zero. Delete removes the full residual. Trade is informational and does not change the reconstruction. Invalid IDs or underflow in an applied effect mark the feed state inconsistent; do not patch it by guessing.

Emit in mutation order

For each execution, emit BookExecution for the maker followed by Trade. At the end, emit BookAdd for a limit residual. Cancel emits BookDelete. A same-price reduction emits BookCancel for the reduction; a no-op emits nothing. A priority-losing amendment emits BookDelete, then any maker executions/trades, then BookAdd if residual rests. Assign one sequence per event, including informational Trade.

Example: ask 10 has 8 at 101, and buy 20 takes 5. Effects are BookExecution(10,5), then Trade(10,20,101,5). The subscriber retains ask 10 for 3. A second decrement on Trade would incorrectly underflow or empty it. Compare the reconstructor and engine after the command's whole effect bundle.

Exercise: how many bytes is a no-trade result? Answer: payload 45, total 57. One trade gives payload 77, total 89. A BookDelete gives header 12 plus payload 8, total 20. Derive at least one golden record for every type; later framing needs those exact lengths.

E03 Separate responses from market data

Prerequisite: E02. The preceding E03 teaching contains the complete custom result and feed payload tables.

Understand the mechanism

A client result reports request outcome. Feed events report visible book effects. Use one bounded result bundle per request and one ordered feed effect per sequence. The custom formats do not claim OUCH or ITCH compatibility.

Work through an example

Ask 10 at 101 has 8. Buy 20 takes 5. The client result contains that execution; the feed emits `BookExecution(10,5)` and then informational `Trade(10,20,101,5)`. The subscriber retains ask 10 for 3 and does not subtract again on `Trade`.

Build it in this order

1. Implement type 0x80 result bundles and the exact payload sizes in the included table. Correlate client request and engine command sequences. Preflight the network variant's 64-maker output cap.
2. Implement `BookAdd`, `BookExecution`, `BookCancel`, `BookDelete`, and `Trade`. Assign one feed sequence to every event, including `Trade`. Emit effects at mutation points.
3. For a priority-losing amendment, publish deletion, maker executions/trades, and any new residual add. For a reduction, publish the size change without moving FIFO.
4. Build a separate `MarketDataBook` that applies events without matching. Compare its full snapshot after each command's effect bundle. Create golden bytes and malformed-input tests for each type.

Check the result

The feed book and engine agree, including FIFO. A no-trade result is 57 bytes including its header. Accepted, rejected, canceled, amended, executed, and expired outcomes follow the included result-bundle rules.

If it fails: check for double subtraction, a residual added before matching ends, or an amendment that failed to update FIFO. Keep the network capacity policy separate from the original unbounded core.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Winsock from startup to an echoed message

Terms introduced here

A socket is an operating-system communication endpoint represented by a handle. Winsock is the Windows socket API. TCP carries an ordered byte stream over a connection; IPv4 identifies endpoints with four-byte network addresses. A port distinguishes services at one address. Loopback addresses refer to the same machine. Blocking means an operation can wait for progress before returning. The peer is the other endpoint.

Use the MinGW-w64 Windows headers with g++. Before those includes, define `WIN32_LEAN_AND_MEAN` to reduce Windows header imports and `NOMINMAX` to avoid min/max preprocessor macros interfering with C++ functions. Include `winsock2.h` before headers that might include the older `winsock.h`, then `ws2tcpip.h`. In CMake, link each network executable with `ws2_32`. The lowercase library name resolves the MinGW import library. For a direct compile, put `-lws2_32` after the source/object arguments.

Own resources with destructors

Resource Acquisition Is Initialization, or RAII, means an object's lifetime controls a resource. A successful socket acquisition creates exactly one owner. The owner closes it on every exit path, including an exception. A borrowed `SOCKET` passed to `send` does not take ownership.

`WSADATA` receives startup information, including `wVersion`, the negotiated API version. `MAKEWORD(2,2)` constructs the requested version value. `WSAStartup` initializes Winsock; `WSACleanup` releases that initialization; `closesocket` releases an owned socket. `std::runtime_error` is a standard exception carrying a message.

```
// Windows-only miniature; include these before other Windows headers.
#define WIN32_LEAN_AND_MEAN
#define NOMINMAX
#include <winsock2.h>
#include <ws2tcpip.h>
#include <stdexcept>
struct Winsock {
    Winsock() {
        WSADATA data{};
        int error = WSAStartup(MAKEWORD(2, 2), &data);
        if (error != 0) throw std::runtime_error("WSAStartup");
        if (data.wVersion != MAKEWORD(2, 2)) {
            WSACleanup();
            throw std::runtime_error("Winsock version");
        }
    }
    ~Winsock() { WSACleanup(); }
    Winsock(const Winsock&) = delete;
    Winsock& operator=(const Winsock&) = delete;
};

struct Socket {
    SOCKET value;
    explicit Socket(SOCKET s) : value(s) {
        if (s == INVALID_SOCKET) throw std::runtime_error("socket");
    }
    ~Socket() { closesocket(value); }
    Socket(const Socket&) = delete;
    Socket& operator=(const Socket&) = delete;
};
```

Declare Winsock before socket owners so it is destroyed last. The example wrappers deliberately disable copying and do not implement movement. Record concrete error codes in the application: WSASStartup returns its error directly; most later failures use WSAGetLastError immediately after the failed operation. [19]

Establish the connection

AF_INET selects IPv4; SOCK_STREAM selects a stream socket; IPPROTO_TCP selects TCP. sockaddr_in is the IPv4 address structure; sockaddr is the generic address view expected by socket calls. htons converts a 16-bit port to network byte order, and inet_pton converts a numeric address string to address bytes. INVALID_SOCKET denotes failed socket acquisition; SOCKET_ERROR denotes failure for calls such as bind/send.

The server creates socket(AF_INET, SOCK_STREAM, IPPROTO_TCP), prepares a zero-initialized sockaddr_in, sets sin_family=AF_INET, sin_port=htons(9000), and calls inet_pton(AF_INET, "127.0.0.1", &address.sin_addr). Require inet_pton to return 1. Call bind, then listen with SOMAXCONN; both return zero on success. accept returns a new connected SOCKET or INVALID_SOCKET. Keep the listening socket and accepted socket as separate owners.

```
// listener owns the created socket; address is the sockaddr_in above.
int result = bind(listener.value,
    reinterpret_cast<const sockaddr*>(&address), sizeof(address));
// Require result == 0 before continuing.
result = listen(listener.value, SOMAXCONN);
// Require result == 0; SOMAXCONN requests a system-selected backlog.
Socket connection{accept(listener.value, nullptr, nullptr)};
```

The pointer cast presents the address structure through the generic API parameter; it is not byte decoding. The two null pointers say you do not need the peer address returned by accept. A backlog is pending connection capacity, not a queue of matching orders.

The client creates its own socket and the same address, then calls connect. A zero result means the blocking connection was established. The server must be listening first. Bind only loopback for this local exercise. Linking failures mentioning Winsock symbols mean the ws2_32 dependency is missing; connection refusal usually means no listener at that endpoint.

Transfer bytes with progress

For a blocking send loop, keep offset=0. Call send(s, data+offset, remaining, 0), with each chunk bounded by INT_MAX. A positive n advances offset by n. SOCKET_ERROR fails with a recorded error code; zero while bytes remain is no progress and must not spin forever. Success means the local stack accepted bytes, not that the peer processed them. [21]

The server repeatedly calls recv into a fixed array. For n>0, echo those n bytes using the send loop. For n==0, the peer ended its sending direction. For SOCKET_ERROR, record the error and terminate this exercise connection. The client sends its test string, calls shutdown(s, SD_SEND), and receives until EOF. Require the concatenated echo to equal the original string. [20]

Exercise: recv returns 2 and then 3 for 'hello'. Answer: echo both chunks and concatenate them at the client. TCP preserves byte order, not your application write boundaries. Test the send loop with a fake transport that accepts at most two bytes per call; real loopback may rarely produce a short write.

N01 Build a Winsock echo exercise

Prerequisite: E02. Suggested work unit: one or two focused sessions.

Understand the mechanism

Winsock exposes Windows networking through SOCKET handles. Initialize it with WSASStartup before using sockets and release it with WSACleanup after sockets close. SOCKET is not a Unix file descriptor; use closesocket and Windows error codes. Start with one blocking loopback connection so operating-system behavior is visible. [19]

Work through an example

A client sends hello. The server may receive he and later llo, or all five bytes together. A successful send may consume fewer bytes than requested. The exercise keeps receiving and sending until the intended byte count has transferred. A TCP recv result of zero means the peer closed its sending side. [20, 21]

```
target_link_libraries(echo_server PRIVATE ws2_32)
target_link_libraries(echo_client PRIVATE ws2_32)
```

Build it in this order

1. Create separate echo_server and echo_client executables. Include winsock2.h before any Windows header that could include winsock.h, then ws2tcpip.h. Link both targets with ws2_32.
2. Initialize Winsock 2.2, create an IPv4 TCP socket, and bind the server to 127.0.0.1 port 9000. Listen and accept one connection; connect the client to that endpoint.
3. Write checked receive and send loops that track transferred bytes. On SOCKET_ERROR, record WSAGetLastError. Put SOCKET ownership in a noncopyable RAI wrapper.
4. Run the client and server in two developer terminals. Test empty shutdown and disconnect during a message. Keep this exercise separate from the matching engine.

Check the result

The client receives the exact echoed bytes for several message sizes. Partial transfer and disconnect paths release all sockets. You can explain why successful send does not mean the receiver has processed the data.

If it fails: If networking fails, check startup, bind address, port conflicts, and the actual error code. Do not replace SOCKET with int or call close. A missing Ws2_32 link dependency causes unresolved Winsock symbols.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Framing, ownership, and a committed request

Terms introduced here

A gateway is the adapter between client sockets and the engine. Dispatch means submitting one complete typed command to the engine. A committed command has taken effect even if its response has not arrived. A request in flight has been sent but has not yet received its result. A reverse map looks up the owner from an engine ID rather than the other way around.

Replace the echo's byte copying with the E02 accumulator. Do not assume one recv equals one command. Parse a complete record, perform session checks, translate the client token to an engine ID, dispatch once, and queue one E03 result bundle.

Give each sequence one meaning

Identifier	Owner and meaning
Client session ID	Server-generated identity for this connection
Request sequence	Starts at 1; increases per client connection
Engine command sequence	Assigned at dispatch across accepted command submissions
Client token	Active order identity within one session
Engine order ID	Unique among active orders across this engine
Feed sequence	Counts emitted feed effects, not commands

Malformed framing or a repeated/skipped client request sequence closes the connection in this first protocol. A well-framed unknown cancellation receives a normal rejection. Two sessions may both use token 5; the gateway maps each pair (session,token) to a different engine ID. Cross-session cancellation never looks up a token in another session.

Keep active mappings consistent

Create a mapping only for a resting residual. Remove it when that order fully fills or is canceled, including when a different client's order fills the maker. Maintain a reverse map from engine ID to session/token for that cleanup. An amendment retaining an ID updates the same active association; if it fills completely, remove it. Never keep a token active merely because its response has not been sent yet.

For generated IDs and session IDs, stop before a counter wraps into a reused value. Request sequences reset only with a new session. A response lost during disconnect does not undo a committed command. This exercise intentionally has no reconnect deduplication promise.

Buffers own bytes until transmission finishes

Encode each response into owned storage and maintain an unsent offset. A short send advances the offset and leaves the suffix queued. Only erase the buffer after all bytes were accepted by the local stack. Never enqueue a pointer into a temporary result or the receive buffer that the next recv overwrites.

Set concrete initial bounds: input accumulator 64 KiB, output queue 1 MiB per client, maximum wire payload 4096, and one request in flight per client for this lesson. E03's 64-maker preflight cap bounds the response. On protocol error or exceeded connection buffer bounds, disconnect and apply the documented cancellation policy.

On disconnect, collect that session's active engine IDs, then submit normal cancellations in a deterministic order such as ascending engine ID. Iterating a map while cancellations erase its entries is unsafe unless the iteration pattern accounts for erasure. Publish all resulting effects. Closed sockets need no further responses, but committed trades remain committed.

Exercise: the server executes a market buy, then the client disconnects before receiving its result. Answer: the execution remains in the log and feed. A reconnect cannot assume that the old request failed. 'No response' describes uncertainty at the client, not rejection by the engine.

N02 Connect a framed client to the engine

Prerequisite: N01 and E03. Suggested work unit: one or two focused sessions.

Understand the mechanism

A TCP gateway accumulates bytes until a complete record exists, validates it, and calls the engine. Wire request sequences belong to a client session; the gateway assigns a separate engine command sequence. Client order tokens also need ownership. Otherwise one client could accidentally cancel another client's order with the same token.

Work through an example

Clients A and B both submit token 5. Store (session A, token 5) and (session B, token 5) as different mappings to engine IDs. A cancellation from A resolves only A's mapping. A fragmented 37-byte NewLimit must invoke the engine once, after all 37 bytes arrive.

Build it in this order

1. Replace echo payload handling with the E02 accumulator. Feed complete records to validation and route typed commands through the old in-process engine API.
2. Assign server-side session IDs and engine order IDs. Track ownership and active token mappings. Remove mappings when orders finish; permit reuse only under the active-ID contract.
3. Return encoded results through a send buffer with an offset. Preserve unsent bytes after short writes. Bound input and output buffers and define a clear disconnect-on-protocol-error rule.
4. For this local exercise, accept one request at a time per client. On disconnect, cancel that client's active orders through normal commands and publish their effects. Document this project-specific cancel-on-disconnect policy.

Check the result

A network request produces the same logical result as direct replay. Tests split each frame at several boundaries, coalesce records, reuse tokens after completion, and reject cross-session cancellation attempts. Closing halfway through a frame never submits a partial command.

If it fails: Do not use the wire request sequence as global arrival order across clients. Do not discard unsent response bytes. If the socket closes after a committed request, its execution still happened; the client cannot assume that a missing response means rejection.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

A readiness loop that cannot be stalled by one client

Terms introduced here

Nonblocking operations return when they cannot progress instead of waiting. Readiness means an operation may make progress now. Polling asks the OS for readiness across a set of sockets. A work budget caps service in one visit; fairness means one connection cannot indefinitely prevent others from progressing. Backpressure slows admission when downstream storage fills.

One gateway thread owns every client socket, its input/output buffers, and its connection state. Readiness polling lets it service several connections without waiting inside a blocking `recv` on one silent client. This remains a single-threaded matcher until the concurrency branch.

Set nonblocking mode and choose events

```
u_long enabled = 1;
if (ioctlsocket(socket_handle, FIONBIO, &enabled) == SOCKET_ERROR)
    report_error(WSAGetLastError()); // Handle failure, do not continue.

WSAPOLLFD entry{};
entry.fd = socket_handle;
entry.events = POLLRDNORM;
if (has_pending_output) entry.events |= POLLWRNORM;
```

Set both the listener and accepted sockets nonblocking. `WSAPoll` receives an array of entries and a timeout in milliseconds. A negative/error result requires `WSAGetLastError`; zero means timeout; a positive result says inspect revents. Request writable events only while bytes are pending or you can spin on an idle writable socket. [22]

A single visit has a work budget

For a readable listener, accept until `WSAEWOULDBLOCK` or the accept budget is exhausted. For a readable client, `recv` bounded chunks; append and decode complete frames. For a writable client, send from its pending offset. Positive byte counts advance progress. `WSAEWOULDBLOCK` means keep state and try on a later visit, not connection failure. Other failures start disconnect cleanup. `POLLERR`, `POLLHUP`, and `POLLNVAL` also require handling; buffered data and `recv` results determine what can still be consumed.

Use initial budgets of 64 KiB or 32 frames per connection per visit and a finite poll timeout such as 20 ms. Rotate the starting connection so a busy first entry cannot monopolize every visit. Keep work budgets configurable; these values are teaching choices, not throughput claims.

Preserve connection identity through removal

Do not retain vector indices across erasure or reallocation of connection records. Collect closed connections for removal after the visit, or use stable session IDs to resolve state. Rebuild the poll array from currently live connections. A reused numeric socket handle must never inherit a previous session's buffered frame or ownership map.

Verify fairness and bounded memory

Client A sends half of a 37-byte frame and pauses. B sends a complete valid order. B must receive a response before A finishes. A later completes and dispatches once. Next, make A stop reading responses while B continues; A reaches its output limit and disconnects under policy, while B still progresses.

Exercise: a socket is reported readable but `recv` returns `WSAEWOULDBLOCK`. Answer: retain the connection and return to polling. The operation result is authoritative. Another exercise: every idle socket requests `POLLWRNORM`. Answer: the loop can use a full CPU despite no useful traffic; enable that event only for pending writes.

N03 Serve multiple clients with readiness polling

Prerequisite: N02. Suggested work unit: one or two focused sessions.

Understand the mechanism

A nonblocking socket returns `WSAEWOULDBLOCK` when an operation cannot proceed now. `WSAPoll` tells you which sockets may make progress; it does not complete their reads or preserve application messages. Keep one gateway thread owning connection buffers and socket calls. This is a readiness loop, not a thread per order. [22]

Work through an example

Client A sends half a frame and pauses. Client B sends a complete order. The gateway buffers A's bytes and still processes B. When A later finishes, its command receives the next engine sequence assigned at dispatch time. This makes the observed processing order explicit.

Build it in this order

1. Set listening and accepted sockets nonblocking using `ioctlsocket` with `FIONBIO`. Store per-connection input bytes, output bytes, send offset, and session identity.
2. Use `WSAPoll` for readable sockets and for writable sockets only when output is pending. Handle accept, recv, and send until they would block, subject to a per-client work budget.
3. Process error and hangup flags, read available buffered data as appropriate, then apply your disconnect policy. Remove closed handles from the poll set without leaving stale indices.
4. Test one stalled client beside two active clients. Cap bytes and frames processed per visit so a busy connection cannot starve others. Add a bounded policy for a client that never reads responses.

Check the result

A partial frame on one connection does not block other clients. Short writes retain offsets. A slow reader is disconnected according to the documented buffer policy rather than growing memory without bound. The matcher still has a single owner.

If it fails: Polling writable sockets with no pending data can create a busy loop. Treating `WSAEWOULDBLOCK` as a fatal error disconnects healthy clients. Readiness can change before the operation, so the operation's return value remains authoritative.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

A UDP envelope and a stale-state rule

Terms introduced here

UDP sends separate messages called datagrams without TCP's reliable ordered stream. An envelope is an outer header around event records. A watermark is the highest position reached under a declared rule. A heartbeat reports liveness and a watermark even without new data. Stale means the subscriber knows continuity is missing. A transport shim is a test adapter that changes packet delivery.

UDP messages are datagrams. A `recvfrom` call returns a datagram rather than a TCP stream prefix. Datagrams can be lost, duplicated, or reordered. Treat a truncated/oversized datagram as invalid and discard it; never feed its remainder to the stream accumulator. The application must establish continuity itself. [23]

Specify the envelope completely

Use UDP port 9001, initially at 127.0.0.1. Integers are big-endian. The complete datagram, including this 24-byte header, must fit 1200 bytes. This budget is a project choice; it is not a guarantee against fragmentation on every network.

Offset	Field	Bytes
0	Magic ASCII LOBF	4
4	Version 1	1
5	Kind: 0 Data, 1 Heartbeat	1
6	Feed session ID	8
14	First event sequence, or heartbeat watermark H	8
22	Event count	2

After the header, Data contains count complete E03 event records. Datagram length defines the payload length; reject leftover bytes, bad counts, or mismatched inner sequences. Count is at least one for Data. A Heartbeat has count=0, no payload, and H equal to the latest published event sequence, with H=0 before the first event. Sequence arithmetic must not wrap. A new publisher session requires a new baseline. This envelope borrows sequencing ideas, not MoldUDP64 compatibility. [24]

Start with one event per datagram. A BookAdd record is 45 bytes including its own header, so that datagram is 69 bytes. Every feed event gets a sequence even if informational; heartbeat transmission does not consume an event sequence.

Decide whether an event can be applied

The subscriber stores session and expected sequence. If an event equals expected, apply it and increment expected. If it is below expected, discard a known duplicate. If it is above expected, mark the book Stale and buffer later events within a fixed limit. A batch that overlaps already-applied events is validated in full and applies only its contiguous unseen suffix. Unknown sessions trigger recovery, not duplicate handling.

If a heartbeat says $H \geq \text{expected}$ and the expected event does not arrive within the declared short reorder timeout, a gap exists even when the lost event was the last data packet. Silence alone does not distinguish no trading from a failed publisher, so track a separate heartbeat timeout.

Make loss reproducible

Put a shim before delivery that can drop a specified sequence, duplicate it, delay it, or swap two packets. Test a clean stream, missing middle event, missing last event, duplicate, reorder, malformed datagram, and changed session. Preserve the injected delivery schedule with each test.

Exercise: expected=41, receive 43. Answer: do not apply 43 to a book missing 41 and 42. Mark Stale and request the snapshot handoff in N05. Receiving a duplicate Trade cannot legitimately consume a new sequence or decrement the book twice.

N04 Publish a sequenced UDP feed

Prerequisite: N03 and E03. Use the complete 24-byte envelope in the preceding N04 teaching.

Understand the mechanism

UDP preserves datagrams but permits loss, duplication, and reordering. Session and sequence identify continuity; timestamps do not replace them. Start with loopback unicast before multicast.

Work through an example

Expected event is 41 and event 43 arrives. Mark the book Stale; do not apply 43 over missing 41 and 42. A changed session always needs a new baseline. Heartbeats expose loss of a final data event.

Build it in this order

1. Implement the specified magic, version, kind, session, sequence/watermark, and count fields. Enforce a 1200-byte total datagram budget. Data payload length follows from datagram length.
2. Send one E03 effect per packet to 127.0.0.1:9001. Implement recvfrom validation separately from the TCP accumulator.
3. Apply only contiguous events, discard verified duplicates, and bound later-event buffering on a gap. Track Stale and heartbeat timeout independently.
4. Inject a specific drop, duplicate, delay, reorder, malformed packet, and session change through a transport shim. Save the delivery schedule with each regression.

Check the result

A clean stream reconstructs the engine. Missing data prevents a Live claim. A heartbeat advertises the latest published event without consuming a sequence. Recovery uses N05.

If it fails: do not apply partial/truncated datagrams or treat Trade as a second decrement. Reject oversized packets. A successful recvfrom does not prove continuity.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Snapshot plus an exact suffix

Terms introduced here

Recovery rebuilds a trustworthy state after continuity is lost. A staging book is a temporary replacement that cannot become visible until validation finishes. A handoff position S means the snapshot includes all effects through S. Catch-up applies a consecutive suffix after that position. Atomic installation here means readers see the old complete snapshot or the new complete snapshot, never a partially loaded mixture.

Recovery replaces an untrusted subscriber state with a complete snapshot and then applies every event after that snapshot position. The state copy and position must describe the same command boundary. A snapshot of some levels before a command and others after it can be impossible even if every individual record looks valid.

Define the request and response records

Use the E01 12-byte header over a separate recovery TCP connection. A client sends type 0x90 with a positive recovery request sequence and zero payload. Responses echo that request sequence. The following payload definitions make the local recovery exchange self-contained; they are not GLIMPSE-compatible. [25]

Type	Payload fields	Bytes
0x91 Start	feed session u64, included feed S u64, order count u64	24
0x92 Order	ID u64, side u8, price u64, remaining u64, priority u64	33
0x93 End	feed session u64, included feed S u64, order count u64	24

Send orders in the canonical snapshot order. Start and End must match and exactly order count records must lie between them. Each record remains under the 4096 payload cap. Validate IDs, sides, prices, quantities, unique IDs, and FIFO order before installation. Incomplete TCP responses cannot replace the live snapshot.

Take the snapshot at the owner

Between commands, the matcher copies the full resting state and records the last assigned feed effect sequence S. In the single-threaded publisher, finish the preceding effect bundle before taking this boundary. In C02, capture S in the same result/snapshot ordering and ensure prior effects retain their sequence during publication. Do not read a publisher counter at one time and copy a mutable book at another.

The subscriber collects the snapshot into a fresh staging book while continuing to buffer UDP within a limit, initially 100,000 events or 16 MiB, whichever comes first. Install the staging book only after valid End, discard buffered events at or below S, and apply a contiguous suffix starting at S+1. If a gap persists or the buffer overflows, discard that attempt and request a newer snapshot.

Keep status meaningful

Live means the subscriber has a consistent state through a known sequence. Stale means a gap or invalid effect has broken continuity. Recovering means a snapshot is being staged. CatchingUp means a complete snapshot is installed and a suffix is being applied. Compare the applied sequence with a recently received heartbeat watermark H; caught up through H is a precise claim, not a promise that no later event exists.

Worked example: lose 41; buffer 43 and 44; install snapshot through S=42. Drop nothing above 42, then apply 43 and 44. If snapshot S=44, discard both. If S=40 and 41 is still absent, remain Stale/Recovering. Never label the same incomplete state Live because a timeout elapsed.

Exercise: a snapshot says S=42 but includes an order added by event 43. Answer: applying 43 afterward duplicates the add. The sequence stamp is wrong even if all counts and checksums pass. Capture state and S together on the state owner.

N05 Recover a feed from a consistent snapshot

Prerequisite: N04. Suggested work unit: one or two focused sessions.

Understand the mechanism

Recovery needs a snapshot and an exact stream position that describe the same state. A snapshot taken while mutation continues can mix old and new levels. For the single-owner version, let the matcher process a snapshot request between commands. Tag the copied state with the latest feed sequence S that it includes.

Work through an example

The subscriber loses event 41 and buffers 43 and 44. A recovery snapshot says it includes through 42. Install that snapshot, discard buffered events at or below 42, then apply 43 and 44 in order. If the snapshot includes through 44 instead, discard both. If 43 is still missing, remain stale.

Build it in this order

1. Add explicit subscriber states Live, Stale, Recovering, and CatchingUp. On a gap, request recovery over TCP while buffering later datagrams within a fixed limit.
2. Have the owning matcher copy orders, remaining quantities, and FIFO order at a command boundary with session ID and sequence S . Encode an explicit snapshot-start and snapshot-end record.
3. Install the completed snapshot atomically in the subscriber, then apply a contiguous buffered suffix after S . If the buffer overflows or a gap remains, discard that attempt and request a newer snapshot.
4. Use a heartbeat carrying the latest published sequence to decide when catch-up reaches the publisher. Test a delayed snapshot while new commands continue, a restart, and a truncated recovery response.

Check the result

The subscriber never labels an incomplete or mixed snapshot Live. A test losing event 41 converges to the engine after snapshot plus suffix. Snapshot creation runs on the state owner, and its sequence describes exactly the effects included.

If it fails: The most common bug is taking a state copy and sequence stamp at different times. In a future sharded system, a single global stamp needs coordination; use per-shard snapshot positions unless you implement a consistent global barrier.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Multicast and batching on native Windows

Terms introduced here

Multicast sends one datagram toward a group of subscribing endpoints. Membership joins that group on a chosen network interface, the adapter used for network traffic. TTL, or time to live, bounds IP forwarding hops. Batching combines several consecutive events into one datagram. A flush sends the pending batch even if its size limit has not been reached.

Multicast changes delivery configuration, not matching or recovery. Keep N04's envelope, session IDs, sequence rules, and N05 recovery. Test two local subscribers before experimenting on a physical network.

Set the socket options with an explicit interface

Choose a private local test group such as 239.255.0.1 and UDP port 9001. Use a local IPv4 interface address returned by `Get-NetIPAddress -AddressFamily IPv4` in PowerShell; the publisher interface and subscriber membership must refer to the intended interface. Do not copy a network address from someone else's example. [26]

Each subscriber creates an IPv4 UDP socket, enables `SO_REUSEADDR` before bind for this two-subscriber exercise, and binds `INADDR_ANY` at port 9001. Build `ip_mreq` with the group in `imr_multiaddr` and chosen local interface address in `imr_interface`, then call `setsockopt` at `IPPROTO_IP` with `IP_ADD_MEMBERSHIP`. Drop membership at teardown or close the socket. Check every return code.

The publisher selects `IP_MULTICAST_IF` using the chosen `in_addr` and sets `IP_MULTICAST_TTL` to 1 for this local-network exercise. Send to the group address and port. In Winsock, `IP_MULTICAST_LOOP` controls the receive path, unlike the common Unix interpretation; configure it on receiving sockets for local-loopback tests. Use `DWORD` values for the Windows boolean/TTL options as documented. Treat a membership failure as a network setup problem, not a matching bug. [26]

Flush by size and by time

Maintain a batch of consecutive event records. Start a deadline when its first event enters. Flush when adding another event would exceed 1200 total datagram bytes, when count reaches a configured maximum, or when the oldest event reaches the deadline. An idle event loop must wake for this deadline. Normal shutdown flushes a final underfilled batch before closing output.

A batch records first sequence and count in the envelope; each inner event must agree with that sequence range. A heartbeat must not advertise an event as published while it is still waiting in an unsent batch. Track the assigned and published watermarks separately if needed.

Include batching delay in the measurement

If events arrive every 100 microseconds and the batch size is four, the first event may wait 300 microseconds for the fourth before send starts. A timer placed only around send hides that delay. Record event creation-to-subscriber-application latency on compatible clocks, send calls, CPU use, and offered event rate. Compare the same workload with one-event packets and a few bounded batch sizes.

Exercise: one event enters a batch, then traffic stops. Answer: the deadline must flush it without waiting for another event. Two subscribers should reach the same state through the same sequence; one subscriber's injected loss and recovery must not alter the other's book.

N06 Add multicast and measure batching

Prerequisite: N05. Suggested work unit: one or two focused sessions.

Understand the mechanism

Multicast lets one UDP publication reach multiple subscribers that join a group. Group membership, network interface, and loopback settings add configuration beyond unicast. They do not change the sequencing and recovery logic you already tested. Batching amortizes packet overhead but may add waiting time before publication.

Work through an example

Two subscribers join the same local test group and reconstruct the same feed. Subscriber B deliberately drops an event and recovers while A stays live. A batch of four events can reduce send calls, but the oldest event waits until either the batch fills or a flush deadline expires.

Build it in this order

1. Select an IPv4 multicast group and local test port. Configure publisher interface, TTL for the intended local scope, and multicast loopback. Join the group with `IP_ADD_MEMBERSHIP` using an explicit interface.
2. Run two subscribers. Reuse the N04 loss injector and N05 recovery path. If multicast configuration fails, keep the unicast tests green while diagnosing the chosen interface and group membership.
3. Add optional bounded batches with both a maximum count and maximum wait. The envelope declares first sequence and count; sequences inside a batch remain contiguous.
4. Compare single-event packets with several batch sizes at a fixed arrival workload. Measure events per second, send calls, CPU use, and event latency including batch waiting.

Check the result

Two subscribers agree on state after the same sequence. One subscriber can recover independently. A final underfilled batch is flushed during normal shutdown and by the deadline during idle periods. Timings state whether transport is loopback or a physical network.

If it fails: Do not diagnose multicast by rewriting matching logic. Check interface selection and membership first. A benchmark that starts its timer only when send is called hides the batching delay it is meant to measure.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Threads, a mutex, and a closeable bounded queue

Terms introduced here

Concurrency lets multiple execution flows overlap. A thread is one such flow within a process. Synchronization establishes safe access and ordering for shared state. A mutex allows one holder at a time. A predicate is a Boolean condition; a condition variable lets a thread wait until it should recheck one. A spurious wakeup is a return from waiting without the desired state change. Joining waits for a thread to finish. A latch or barrier is a coordination object used to arrange test stages.

A thread is another execution flow in the same process. It shares memory with other threads. Unsynchronized conflicting access to an ordinary object, with at least one write, can be a data race and undefined behavior. A mutex gives exclusive access and orders protected work between owners. A condition variable releases that mutex while a thread waits and reacquires it before the predicate is checked again. [27]

Define the queue's state machine

The queue owns fixed storage, a head index, item count, capacity, and closed flag. The mutex protects all of them. Push waits for space or closure; Pop waits for data or closure. Close rejects later pushes, wakes all waiters, and lets consumers drain queued data before returning empty/closed. Capacity zero is a construction error.

`std::unique_lock` owns a mutex lock and can unlock/relock it for a condition-variable wait. `std::lock_guard` holds a lock for its scope. `notify_one` wakes one waiter; `notify_all` wakes all, and every awakened thread rechecks its predicate. `std::invalid_argument` is a standard exception indicating invalid caller input. The following miniature uses `uint64_t` values so object construction cannot distract from synchronization. Save it as a standalone exercise before making a generic queue. Its destructor is not a stop signal: close and join all users before destroying it.

```
#include <condition_variable>
#include <cstdint>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <vector>

class BoundedQueue {
    std::vector<std::uint64_t> slots;
    std::size_t head = 0, count = 0;
    bool closed = false;
    std::mutex mutex;
    std::condition_variable readable, writable;
public:
    explicit BoundedQueue(std::size_t capacity) : slots(capacity) {
        if (capacity == 0) throw std::invalid_argument("capacity");
    }
    bool push(std::uint64_t value) {
        std::unique_lock lock(mutex);
        writable.wait(lock, [&] {
            return closed || count < slots.size();
        });
        if (closed) return false;
        slots[(head + count) % slots.size()] = value;
        ++count;
        lock.unlock();
        readable.notify_one();
    }
```

```

        return true;
    }
    std::optional<std::uint64_t> pop() {
        std::unique_lock lock(mutex);
        readable.wait(lock, [&] { return closed || count != 0; });
        if (count == 0) return std::nullopt;
        auto value = slots[head];
        head = (head + 1) % slots.size();
        --count;
        lock.unlock();
        writable.notify_one();
        return value;
    }
    void close() {
        {
            std::lock_guard lock(mutex);
            closed = true;
        }
        readable.notify_all();
        writable.notify_all();
    }
};

```

Follow one blocked push

Capacity two holds A and B. The producer attempts C, sees `count==capacity` under the mutex, and waits. Waiting releases the mutex, allowing the consumer to pop A. The consumer changes count, unlocks, and notifies writable. The producer reacquires the mutex and checks the predicate again before writing C. A wakeup alone never grants permission; another waiter or a spurious wakeup may make the condition false.

If Close happens while the producer waits, the predicate becomes true because closed is true; push returns false. A consumer waking from Close still takes queued items, then returns nullopt once count reaches zero. That distinction provides drain-on-close behavior.

Run and test thread ownership

Use `std::thread producer([&]{...})` and `consumer([&]{...})`; join both before local queue destruction. Send integers 0 through 9999, close after the last successful push, and require the consumer's observed vector to equal that sequence. Access that vector from the parent only after `consumer.join()`. If a test fails inside a worker, capture a failure flag/value and assert after joins instead of allowing an exception to leave a joinable thread unhandled.

For deterministic blocked-state tests, add a test-only hook immediately before the predicate wait releases the lock; signal a latch so the test can arrange Close or Pop. Do not use an arbitrary sleep as proof that a worker reached the intended state. Use test timeouts only to diagnose a hang, not to define correctness.

Exercise: pop sees `closed=true` and `count=1`. Answer: return the queued item, not nullopt. A predicate of `count>0` without closed would leave a consumer stuck forever after closing an empty queue. Passing stress tests supports the implementation, but the mutex/predicate argument explains why it works.

C01 Build a bounded queue with a mutex

Prerequisite: L12 and basic thread syntax. Suggested work unit: one or two focused sessions.

Understand the mechanism

A bounded queue has both storage and a state machine. A producer waits when it is full; a consumer waits when it is empty. A mutex protects the queue state. A condition variable avoids spinning while a predicate is false. Notifications are hints to recheck a predicate, not a replacement for that predicate.

Work through an example

A capacity-two queue receives A and B. Pushing C must wait or return Full according to the API. After the consumer removes A, C can enter. Closing a queue rejects future pushes, wakes both sides, lets the consumer drain existing items, and returns Closed once empty.

Build it in this order

1. Write push, pop, and close using a mutex, two condition variables, and fixed capacity. Define whether push blocks; use the blocking policy for this exercise.
2. Wait with predicates that include closed. After a state change, release the lock before notifying where appropriate. Define close as idempotent and wake all waiting producers and consumers.
3. Test empty/full transitions and order preservation with capacity one and two. Use barriers or explicit handshakes to arrange waits rather than relying on sleeps.
4. Send a known sequence of unique numbers through one producer and consumer, close after the final item, drain, and join. Keep this queue independent of sockets and the engine.

Check the result

The consumer gets each item once and in order. Closing while full or empty releases waiters and terminates both threads. Zero capacity is rejected by construction. The synchronization explanation names the mutex-protected state and wait predicates.

If it fails: A missed closed check can deadlock shutdown. A bare wait without a predicate can misbehave after a spurious wakeup. A stress run is evidence, but not proof of all interleavings.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Assign owners before adding concurrency

Terms introduced here

Ingress is the incoming command path; downstream is the next consumer of output. Exclusive ownership means exactly one component may mutate a resource. `unique_ptr` transfers one owned object; `shared_ptr` shares its lifetime but does not synchronize its mutations. SPSC means single producer, single consumer: one thread supplies items and one removes them on that queue. Deadlock is a cycle of waits in which no participant can make the progress another needs.

Move values between threads; do not let several threads manipulate the book or the same socket. Ownership is the primary rule. A queue implementation only supplies a safe handoff when its producer/consumer topology and object lifetimes match the design.

A complete first ownership map

Owner	Mutable state	Receives	Sends
Gateway	Client sockets, buffers, session/token maps	Client bytes, result bundles	Typed commands
Matcher	Order book, engine sequence, effect generation	Commands	Results and feed-effect bundles
Publisher	Feed socket, pending batches, published watermark	Feed-effect bundles	UDP datagrams

Use one gateway-to-matcher queue, one matcher-to-gateway queue, and one matcher-to-publisher queue. Each has one producer and one consumer. The gateway updates active mappings from ordered results/effects routed to it; it cannot inspect live book state directly. A snapshot request is a message to the matcher, which copies at a command boundary and transfers owned snapshot data.

A queue of shared pointers is not enough if the producer keeps editing the pointed-to object. Transfer a value or `unique_ptr` and stop using it on the sender. The receiver owns the buffer until its work and transmission are finished. Never queue a span into a temporary parser buffer.

Avoid a cycle of blocked owners

Suppose ingress is full, so the gateway blocks trying to push. Meanwhile the matcher cannot publish a result because the response queue is full. Only the blocked gateway can drain that queue. Both threads are waiting for each other. Use `try_push` for gateway admission; while ingress is full, keep draining responses and polling existing output. Backpressure should pause new acceptance, not prevent required downstream progress.

The first matcher can block publishing to its output queues if their consumers are guaranteed to keep draining and no return dependency creates a cycle. Reserve the capacity needed for an accepted command's bounded output, or retain a complete pending result bundle until delivered. Never silently drop an accepted execution because a queue filled.

Shut down in dependency order

Stop admitting new requests. Finish any already-dispatched commands and generate deterministic cancel-on-disconnect commands while the gateway can still send ingress. Close ingress only after those final commands. The matcher drains ingress, then closes its result and publication queues. The gateway and publisher drain those outputs, flush final bytes/batches under a declared timeout policy, then terminate. Join every thread before destroying queues, sockets, or book resources.

A stop flag alone cannot wake a condition-variable wait. Queue Close wakes its waiters. A gateway blocked indefinitely in an OS poll also needs a finite timeout or an explicit wakeup channel. For forced shutdown after a timeout, report unsent responses; do not rewrite committed engine history.

Exercise: you give one matcher-output SPSC queue to both gateway and publisher. Answer: it now has two consumers, violating the queue's premise and splitting messages unpredictably. Use separate queues or a sole dispatcher. Add one thread at a time and preserve the direct in-process replay path for core tests.

C02 Move work between owning threads

Prerequisite: C01 and E03 with an optional N03 gateway. Suggested work unit: one or two focused sessions.

Understand the mechanism

Concurrency starts by assigning state to owners. The matcher alone owns the order book. The gateway owns client sockets and connection buffers. A publisher owns its output socket. Threads transfer values or exclusive ownership through queues, so the consumer never observes a buffer that the producer is still editing.

Work through an example

The gateway decodes command 20 and enqueues a value. The matcher consumes it and produces an immutable result bundle. A single outgoing SPSC edge cannot have both the gateway and publisher as consumers. Give client responses and feed publications separate queues, or use one dispatcher that is the sole consumer.

Build it in this order

1. First move publication to one thread using the mutex queue. Measure before adding another thread. Then move ingress if blocking or independent work gives it a reason to exist.
2. Write an ownership table for sockets, connection maps, books, queue ends, and output buffers. Route client responses back to the socket-owning gateway through a separate queue.
3. Define bounded backpressure. Start by stopping new admission when downstream capacity is exhausted, while the gateway continues draining responses. Do not silently discard accepted executions or feed effects.
4. Implement shutdown: stop admission, close ingress, drain the matcher, close its output queues, drain output owners, then join. Use finite poll waits or a wakeup signal so owners can observe stop requests.

Check the result

The old in-process replay still works. Queue-full and shutdown tests complete without deadlock, lost results, or socket access from the wrong owner. No thread reads live book state without an explicit snapshot transfer.

If it fails: If the gateway blocks pushing ingress while it is also the only thread that can drain replies, you can create a cycle of full queues. Use a try-push admission path or stage work so every owner can keep making progress.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Build a ring with one producer and one consumer

Terms introduced here

A ring is a fixed array used cyclically; a cursor identifies the next read or write position. An atomic operation accesses its atomic object indivisibly relative to other atomic operations on it. Sequential consistency (seq_cst) gives those operations a single order satisfying the model's ordering constraints. Lock-free means the atomic implementation does not use a blocking lock; check support rather than infer it from the ring's name.

SPSC means exactly one producer calls `try_push` and exactly one consumer calls `try_pop`. Each owns one cursor. An array of `N` slots with one reserved slot holds `N-1` values. Empty means `read==write`; full means `next(write)==read`. Start with sequentially consistent atomic cursors and plain `uint64_t` payloads.

A complete minimal ring

```
#include <array>
#include <atomic>
#include <cstddef>
#include <cstdint>
#include <optional>

template<std::size_t N>
class Ring {
    static_assert(N >= 2);
    std::array<std::uint64_t, N> slots{};
    std::atomic<std::size_t> read{0}, write{0};
    static std::size_t next(std::size_t i) { return (i + 1) % N; }
public:
    bool try_push(std::uint64_t value) {
        auto w = write.load();
        auto n = next(w);
        if (n == read.load()) return false;
        slots[w] = value;
        write.store(n);
        return true;
    }
    std::optional<std::uint64_t> try_pop() {
        auto r = read.load();
        if (r == write.load()) return std::nullopt;
        auto value = slots[r];
        read.store(next(r));
        return value;
    }
};
```

The default load/store ordering is `seq_cst`. The atomics do not make every payload access atomic; the cursor handoffs order when each ordinary slot may be written or read. C04 explains both directions before weakening those operations. [28]

Follow wraparound by hand

With $N=4$, start $r=w=0$. Push A into 0, B into 1, C into 2; w becomes 3 and the ring is full. Pop A, moving r to 1. Push D into slot 3, moving w to 0. Pop order remains B, C, D even though physical indices wrap. No cursor needs to grow without bound.

If `try_push` fails, the caller retains the value and may retry. If `try_pop` returns `nullopt`, it has not consumed anything. Neither failed operation should modify its cursor. A full ring is normal backpressure, not an instruction to overwrite the oldest element.

Completion is a separate handoff

Use `atomic<bool> finished=false`. The producer stores true only after its final successful push. The consumer repeatedly pops. If a pop fails and it observes `finished=true`, it must `try_pop` once more before concluding the stream is drained. This recheck covers a last push that occurred between the initial empty observation and the finished observation. Start with `seq_cst` for `finished` too; C04 can use `release/acquire`.

Exercise: $N=2$. Answer: usable capacity is one. A second push fails until one pop succeeds. Test $N=2$ and $N=4$, millions of wraps, full refusal, empty refusal, and final-item shutdown. Use the same busy-wait/yield/sleep policy when comparing the mutex queue and ring, or explicitly measure the waiting-policy difference. Atomic cursor operations being lock-free on one build is a property to record, not a claim guaranteed by the class name.

C03 Implement a single producer ring

Prerequisite: C01 and C02 ownership model. Suggested work unit: one or two focused sessions.

Understand the mechanism

An SPSC ring works because only one producer writes the write cursor and only one consumer writes the read cursor. Reserve one slot to distinguish full from empty: an array of N slots holds at most N minus 1 items. Start with atomic cursors using sequential consistency and a trivial fixed-size value type.

Work through an example

For $N = 4$, $\text{read} = \text{write} = 0$ means empty. After pushes A, B, C, write is 3 and the ring is full because the next write index would equal read. Pop A to move read to 1; pushing D uses slot 3 and wraps write to 0. The remaining pop order is B, C, D.

Build it in this order

1. Write a ring exercise for `uint64_t` values with `try_push` and `try_pop`. Use modulo arithmetic first and reject N below two. Initialize every slot before publishing its cursor.
2. On push, load read and write, test `next(write) != read`, store the payload, then publish write. On pop, check `read != write`, read the payload, then publish read. Use `seq_cst` for both atomic cursors initially.
3. Test capacities one and three usable items, repeated wraparound, full refusal without overwrite, and empty refusal without cursor movement. Synchronize thread start and verify an exact numbered sequence.
4. Define a producer-finished flag published after its last successful push. The consumer exits only after observing completion and rechecking the ring for remaining items. Benchmark this queue separately from engine work.

Check the result

Every transferred number appears once and in order after many wraparounds. You can explain why a slot cannot be reused until the consumer advances read. Record `is_lock_free` for the cursor atomics if you make a lock-free claim; SPSC topology alone is not that claim.

If it fails: A non-atomic cursor shared across threads is not safe. A full/empty off-by-one can overwrite unread data. Do not add arbitrary object construction and destruction until the trivial-value ring is understood.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Why acquire and release publish a slot

Terms introduced here

Release is a publication operation; acquire is the corresponding observation operation used to order preceding writes with later reads. Synchronizes-with names a qualifying cross-thread relationship. Happens-before combines within-thread sequencing and synchronization into an ordering guarantee. Relaxed atomic access preserves atomicity without this publication relationship. The example below states the precise pairs used.

Memory ordering is a relationship between actions. It is not a command to 'flush all caches.' You need one relationship to publish an initialized payload and another to say the consumer is finished before the producer overwrites that slot. Relaxed atomic cursors alone provide indivisible cursor accesses but do not establish those payload relationships. [27, 28]

The two handoffs

Direction	Actions in owner order	Observing action	What becomes safe
Producer to consumer	Write slot, then release-store write cursor	Consumer acquire-loads that publication	Consumer reads initialized slot
Consumer to producer	Read slot, then release-store read cursor	Producer acquire-loads that advancement	Producer reuses the slot

Sequenced-before describes order within one thread. A release operation synchronizes with an acquire operation that reads the required publication. Combining those relationships creates happens-before. For every reuse of a non-atomic slot, establish the consumer's previous read happens before the producer's next write. Merely saying 'the write cursor is atomic' omits half the argument.

Change the existing ring, one operation at a time

```
// try_push body, using Ring's existing members:
auto w = write.load(std::memory_order_relaxed);
auto n = next(w);
if (n == read.load(std::memory_order_acquire)) return false;
slots[w] = value;
write.store(n, std::memory_order_release);
return true;

// try_pop body:
auto r = read.load(std::memory_order_relaxed);
if (r == write.load(std::memory_order_acquire)) return std::nullopt;
auto value = slots[r];
read.store(next(r), std::memory_order_release);
return value;
```

Only the producer writes write, and only the consumer writes read. That single-writer invariant justifies relaxed access to the owner's own cursor in this design. The cross-thread cursor loads remain acquire. Keep the original seq_cst variant for a clear behavioral and performance comparison. Do not add cursor caching or arbitrary nontrivial object construction during this change.

For finished, a release-store after the final push and an acquire-load in the consumer publish completion. Retain the drain recheck from C03. A flag cannot substitute for a correct ring handoff during earlier transfers.

Test evidence and the argument play different roles

Repeat the exact sequence, full/empty, wraparound, and shutdown tests. x64 stress success does not prove the C++ memory model argument; hardware may conceal a missing ordering edge. Conversely, a correct publication argument does not detect an off-by-one index bug. Keep both checks.

Exercise: the producer release-stores write, but the consumer reads write with relaxed. Answer: the required acquire observation is missing, so the ordinary payload read lacks this publication guarantee. Another exercise: the consumer advances read before copying the payload. Answer: the producer may overwrite a slot the consumer still needs. Keep payload use before the release of read.

C04 Explain publication with acquire and release

Prerequisite: C03. Suggested work unit: one or two focused sessions.

Understand the mechanism

Acquire and release let you state the visibility relationship you need. The producer writes a payload before a release-store of the write cursor. A consumer acquire-load that observes that publication can read the initialized payload. In the reverse direction, the consumer releases read after finishing the slot, and the producer acquires read before reusing it. Both directions matter. [28]

Work through an example

Slot 2 is reused many times. Ordering only the producer write cursor protects the consumer's initial read, but does not by itself explain when the producer may overwrite the slot again. The consumer's read-cursor publication supplies that second handoff. Draw those two relationships for one slot before changing code.

Build it in this order

1. Write the payload-store, write-cursor publication, payload-read, and read-cursor publication order on paper. Identify the release/acquire pair that protects each ordinary payload access.
2. Change one reviewed ring variant to release stores for published cursors and acquire loads of the other thread's cursor. Own-cursor accesses may use relaxed order once the single-writer invariant is documented.
3. Repeat the exact sequence, wraparound, full/empty, and shutdown tests. Keep the `seq_cst` implementation as a behavioral and performance reference.
4. Compare under equal waiting policies. If experimenting with cache-line separation of cursors, make that a different change and record object size as well as performance.

Check the result

The explanation covers initial publication and later slot reuse. Every non-atomic shared payload access has an ordering argument. Passing x64 stress tests does not establish portable C++ correctness; keep the proof alongside the test evidence.

If it fails: Relaxed atomics make cursor accesses atomic but do not establish payload publication on their own. `volatile` is not synchronization. If you cannot explain a proposed weaker ordering, retain `seq_cst`.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Shard whole books and define global order

Terms introduced here

Sharding assigns separate books to separate owners. A local sequence belongs to one shard; a global sequence orders a merged output stream. A barrier coordinates participants at a declared boundary. A watermark identifies how far each participant has completed work. A hot symbol receives most of the traffic and can limit scaling even when other shards are idle.

A shard is a matching owner responsible for a fixed set of symbols. Start with TESTA and TESTB on shard 0, TESTC on shard 1. The gateway routes by an explicit table. Never change a live symbol's owner without a migration protocol; never split one book's mutations between two threads.

Separate local order from merged publication

Each symbol receives commands in its declared input order. Each shard produces an ordered effect stream. One publisher consumes one output queue per shard and assigns the global publication sequence in the order it actually merges them. A result should retain symbol, shard/local command position, and client correlation separately from the global feed position.

If two symbols are independent, replay can reproduce their exact individual histories even when the publisher merges them differently on another run. To promise identical global bytes, persist the actual merge choices or implement a deterministic merge with a rule for waiting for missing shard output. A global sequence assigned before parallel matching is not automatically completion order.

Queue topology stays explicit

Use one gateway-to-shard queue and one shard-to-publisher queue per shard. Do not have multiple shards push into an SPSC output ring. The publisher is the sole consumer of each of its input rings. Its round-robin draining policy needs bounded work so one hot shard does not starve another. Track the hot-symbol case as well as balanced traffic when measuring scaling.

For multi-symbol wire traffic, define a new protocol version and a u32 symbol ID prefix for command and feed payloads; increase exact payload lengths by four. Keep a configured symbol-ID table for the session. Reject unsupported versions rather than interpreting old records with shifted offsets. A snapshot identifies the same symbol configuration.

A global snapshot requires a barrier

The simplest consistent global snapshot pauses new dispatch, drains every shard through a declared input watermark, drains and sequences all resulting feed effects, then captures the books and the publisher's global S while shards remain quiescent. Resume admission after the copies and stamp are complete. This costs a pause but establishes one state boundary.

A more advanced alternative uses per-shard watermarks and explicitly records their relationship to feed publication. Do not stamp unrelated local snapshots with one current global counter and call that consistent.

Exercise: two shards publish into one C03 ring. Answer: there are now two producers, so the single-writer cursor proof no longer applies. Give each shard a ring or choose a queue designed and tested for multiple producers. Report speedup under balanced and hot-symbol workloads; a single busy symbol still has one matching owner.

C05 Partition symbols across matching owners

Prerequisite: C02 and optionally C04. Suggested work unit: one or two focused sessions.

Understand the mechanism

A symbol belongs to one matching shard, so its commands have one serial processing order. Different symbols can progress independently. Global publication order is a separate problem from per-symbol matching order. Keep explicit per-shard inputs and outputs so an SPSC queue never silently becomes multiple-producer.

Work through an example

Route TESTA and TESTB to shard 0, and TESTC to shard 1 using a configured table. Each shard produces local sequence numbers. A publisher drains one queue per shard and assigns the final feed sequence in its observed merge order. A replay may preserve each symbol's outputs while producing a different cross-symbol merge.

Build it in this order

1. Add a symbol field and explicit routing configuration. Extend the reference to multiple independent books before adding another matching thread.
2. Create one ingress-to-shard queue and one shard-to-publisher queue per shard. Use a deterministic per-symbol command sequence and never split one book between matchers.
3. Let one publication owner assign the global feed sequence. Persist its actual merge if byte-identical global replay is required; otherwise compare canonical per-symbol outputs.
4. Benchmark balanced symbols and one hot symbol. Test shutdown with one busy and one idle shard. For snapshots, coordinate a barrier or use per-shard watermarks; do not stamp unrelated snapshots with one unproven global position.

Check the result

Each symbol receives the same ordered commands and produces the same result history as its single-thread reference. Queue producer and consumer counts are documented. Report scaling and the hot-symbol limit without implying linear speedup.

If it fails: A shared output ring with several shard producers is no longer SPSC. Hash routing that changes during a run can move a live book to another owner. A global sequence assigned before independently scheduled matching is not automatically global completion order.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Reconstruct a defined ITCH subset

Terms introduced here

ITCH is Nasdaq's market-data message family. BX names the chosen venue. Attribution identifies a market participant associated with a displayed order. ASCII maps text characters to byte values. Fixed-point prices use integers with a declared decimal scale. A transport sequence belongs to delivery framing, while a message timestamp is a time value. This lesson defines the supported subset rather than assuming all market-data messages have the same fields.

ITCH is an output feed: apply exchange events to a reconstruction book, not to the matching-command API. This exercise uses NASDAQ BX TotalView-ITCH 5.0, specification dated 5 December 2014. The tables below contain the fields needed for a visible-order lifecycle fixture. They are enough to complete the exercise without downloading a historical feed. [18]

Read the common prefix and exact widths

Offsets are zero-based from the message-type byte, excluding any transport framing. All integer fields are unsigned big-endian. Common fields: type at 0 (1 byte), stock locate at 1 (2), tracking number at 3 (2), timestamp at 5 (6). Timestamp is nanoseconds since midnight; it is not a transport sequence. Keep prices as integer units of 1/10,000 and preserve the exact stock string, trimming only its trailing spaces for display.

Type / length	Lifecycle fields as offset:width	Application
A / 36	order 11:8; side 19:1; shares 20:4; stock 24:8; price 32:4	Add visible order; side ASCII B/S
F / 40	A fields; attribution 36:4	Add with attribution
E / 31	order 11:8; executed 19:4; match 23:8	Subtract displayed execution
C / 36	E fields; printable 31:1; execution price 32:4	Subtract execution; keep trade metadata
X / 23	order 11:8; canceled 19:4	Reduce displayed remaining
D / 19	order 11:8	Remove full remaining order
U / 35	old ID 11:8; new ID 19:8; shares 27:4; price 31:4	Remove old and add replacement

For U, retain stock, side, and attribution from the old order; the new reference replaces the old one. For C, the printable flag affects trade reporting, not whether displayed quantity is reduced. An informational non-displayed Trade must not subtract an unrelated visible order. [18]

Build fixtures instead of depending on a data download

Use a test-only file envelope with a two-byte big-endian message length followed by that many ITCH bytes. This is the fixture container for this lesson, not a claim about every vendor's historical-file format. E02's checked reads can handle widths 2, 4, 6, and 8. Validate the message-specific length before reading offsets. Use stock-locate 1, tracking 0, increasing synthetic timestamps, and the padded stock 'TEST'.

Construct A for order 10, sell, 8 shares, price 1,010,000 (101.0000). Its first byte is ASCII A (0x41); its last four bytes are 00 0F 69 50. Construct E for order 10, executed 5, match 1. Remaining is 3. Construct X canceling 1, leaving 2. Construct U replacing 10 with 11, remaining 4, price 1,020,000; the new order is sell and old ID 10 disappears. Finish with D for 11; the book is empty.

Keep the ITCH reconstructor's domain separate from the simulator's one-billion-unit per-command policy. Real message quantities are not validated by reusing the custom order-entry validator. The fixture subset can initialize instrument metadata from A/F; a fuller feed loader needs its own directory and session handling.

State the reconstruction boundary

This lesson proves the supported synthetic lifecycle subset. Stop on unsupported message types in the fixture runner. If extending to a real feed, add its framing and every state-relevant message before claiming a complete visible book. Do not silently ignore an unknown type and label the result complete. Detect transport gaps outside ITCH; receive order supplies the event ordering for the fixture.

Exercise: a C message is marked non-printable. Answer: still decrement the identified visible order's executed quantity; avoid treating the printable flag as permission to skip book maintenance. A U message has no side field because the old order supplies it. These are feed semantics, not your simulator's same-ID amendment rule.

X01 Reconstruct a market data book from ITCH

Prerequisite: E03 and E02. Suggested work unit: one or two focused sessions.

Understand the mechanism

Historical ITCH contains exchange-generated market-data events. It can reconstruct the observable book described by supported messages. It is not the original client-command stream and should not be fed into your matcher as if you were reproducing the exchange's matching decisions. Real feeds also contain messages outside the basic visible lifecycle. [18]

Work through an example

An Add says order 10 became visible at 101 for 8. An Executed message says 5 executed, so the reconstruction leaves 3. You apply that event directly. Running a new buy through your matcher to manufacture the same execution would invent an input that was not present in the feed.

Build it in this order

1. Choose one named venue and exact specification version. Implement only Add, Execute, Cancel, Delete, and Replace first, plus the metadata needed to identify instruments.
2. Create byte fixtures from the specification and small hand-written lifecycle examples. Keep the parser separate from MarketDataBook application logic.
3. Define a policy for unsupported messages. Ignore only types shown to be irrelevant to the state you claim to reconstruct; otherwise stop or mark the reconstruction incomplete.
4. Add feed-gap detection at the transport layer. Distinguish ITCH timestamps from the transport's sequence numbers and snapshot recovery position.

Check the result

Supported fixtures reconstruct exact expected orders and depth. Documentation names the venue, format, supported subset, and limits. The matcher and market-data reconstructor remain separate components.

If it fails: A Trade message may not correspond to a visible resting-order decrement. A price or quantity field can differ between message types. Verify exact offsets and semantics instead of sharing a cast across types.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Clean restart, journaling, and a durability boundary

Terms introduced here

Persistence stores data beyond the process lifetime. A checkpoint is a saved complete state at a boundary. A journal records ordered commands after or around checkpoints. Write-ahead means recording a command before applying it. A checksum detects accidental byte changes; CRC is cyclic redundancy check. Durability states what persists after failure. Acknowledgement tells a client a request reached the promised boundary; deduplication recognizes an already handled request.

A clean restart restores the exact logical engine state from a completed file. Crash durability additionally says which acknowledged work survives termination or power loss. Begin with clean shutdown and keep the stronger promise as a separate experiment. Neither a readable replay log nor a call to `ostream.flush()` alone establishes disk durability.

A complete single-symbol checkpoint format

Use unsigned big-endian fields; serialize values, never pointers or iterator bytes. Start with magic ASCII LOBC, u16 version=1, u64 engine session, u64 last processed command, u64 next arrival priority, u64 last assigned feed sequence, then configuration: u64 max price, max quantity, max live orders, and max maker fills (zero means unbounded for those capacity limits). Follow with u64 order count. Each order stores u64 ID, u8 side, u64 price, u64 remaining, u64 priority, for 33 bytes. Preserve canonical FIFO order.

End the file with a four-byte CRC-32 over all preceding file bytes. Define CRC-32 as reflected polynomial 0xEDB88320, initial state 0xFFFFFFFF, and final XOR 0xFFFFFFFF. For each byte, XOR it into the low eight bits, then repeat eight times: shift right once and XOR the polynomial if the old low bit was 1. The ASCII check string '123456789' produces 0xCBF43926. This detects accidental corruption; it is not authentication.

Check the file length, version, configuration, checksum, count, unique IDs, valid ranges, FIFO, and uncrossed state before installation. Ensure next arrival priority exceeds every restored priority. Rebuild the ID locator index from the new containers. Do not restore stale slot handles or pointers from the old instance.

Define the journal around existing command bytes

Start a journal with ASCII LOBJ, u16 version=1, u64 engine session. Each record is u32 record length, one complete E01 command record, then u32 CRC-32 of that command record. The length counts command bytes only, so it is bounded by 4108. The E01 sequence is the engine command sequence. Preserve every submitted command, including commands that yield ordinary rejections. Parsing errors that never reached the engine are not commands.

At restart, load checkpoint through S, validate the journal's session/configuration context, skip already-checkpointed records through S, and replay a contiguous suffix starting S+1. A torn final record may be discarded under the declared recovery policy; a checksum failure in an interior record stops recovery. Do not scan forward for a plausible new boundary and silently skip a committed command.

Work one restart by hand

Checkpoint after command 100. Journal contains complete commands 101 and 102 and half of 103. Restore the checkpoint, replay 101 and 102, discard the torn final 103, and set the next sequence to 103. Compare state and replayed outputs against an uninterrupted run through 102. If your acknowledgement policy had already acknowledged 103, this outcome would violate that policy.

Add crash behavior only after the clean case works

For a write-ahead variant, validate syntax and assign a sequence, append the complete command record, flush according to the declared durability boundary, then process and acknowledge it. A durable command may be replayed after a crash that occurred before its result was delivered. Log configuration and deterministic policies so replay makes the same decisions. A complete unacknowledged record may legitimately be applied after restart; clients need deduplication/result recovery before you can promise exactly-once requests across reconnects.

On Windows, HANDLE is an opaque file-resource identifier owned until CloseHandle. CreateFileW opens a wide-character path; for a new temporary checkpoint, request GENERIC_WRITE, no sharing, CREATE_NEW, and FILE_ATTRIBUTE_NORMAL. Reject INVALID_HANDLE_VALUE. WriteFile takes the handle, data pointer, requested byte count, a DWORD output count, and a null final parameter for synchronous writes. A DWORD is a 32-bit Windows unsigned integer; split larger data into bounded chunks. Check both the Boolean success result and forward progress before advancing the offset. Use GetLastError after failure. These parameter definitions follow [CreateFileW](#) and [WriteFile](#).

Call FlushFileBuffers and check success, then close the temporary file. Keep an older named valid checkpoint. Install the new temporary file in the same directory with [MoveFileExW](#) and MOVEFILE_REPLACE_EXISTING; this flag allows replacement of the destination. Check its return value before publishing that checkpoint name as current. Preserve the old checkpoint and journal until the new one is validated. This is a concrete file workflow to fault-test, not a claim that a rename alone proves power-loss durability. Test on the filesystem you claim to support; file replacement, stream flushing, device caches, and power failure are distinct parts of the guarantee. [29]

Exercise: a snapshot contains depth totals but no individual order IDs or priorities. Answer: it cannot restore FIFO matching. Another exercise: crash after durable journal append but before acknowledgement. Answer: recovery may apply the command; lack of acknowledgement does not prove it was absent. Label the completed initial lesson 'clean restart' until the stronger boundary has its own fault tests.

X02 Restart from a journal and snapshot

Prerequisite: L11 and E02 with optional N05. Suggested work unit: one or two focused sessions.

Understand the mechanism

Restart recovery restores the engine's own state, including FIFO priority, ID policy, configuration, and next sequence. A readable log, a durable journal, and a checkpoint are different promises. Start with clean-shutdown persistence; crash durability needs a declared write and acknowledgement boundary.

Work through an example

A checkpoint includes commands through 100. On restart, load its orders in recorded FIFO order and replay commands 101 onward. A truncated final journal record must not be decoded as a valid command. Replaying 100 again can duplicate effects even if its order ID is no longer active.

Build it in this order

1. Save an explicit versioned checkpoint at a command boundary. Include configuration, sequence, resting orders, remaining quantities, and priority order. Serialize canonical fields rather than pointers or raw structs.
2. Write a journal with length-checked records and a documented integrity check. Record every processed command needed to reproduce results, including rejected-command sequence positions.
3. Restore into a fresh instance, then replay only the suffix after the checkpoint sequence. Compare all outputs and state with an uninterrupted run.
4. For a later crash-durable variant, define when a command is acknowledged relative to journal persistence and use the appropriate Windows flush and file-replacement APIs. Test process termination at each boundary before claiming crash recovery.

Check the result

Clean restart reproduces state and next sequence. A partial trailing record has a defined recovery policy. The checkpoint and suffix agree on format, session, and configuration. Claims distinguish clean restart from crash durability.

If it fails: Reconstructing only aggregate depth loses FIFO. Flushing a C++ stream does not by itself establish the chosen disk-durability guarantee. A checkpoint copied during mutation can preserve an impossible state.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Finish a release another person can reproduce

Terms introduced here

A release is a named, reproducible project state with a declared feature set. A commit records repository content; a tag names a selected commit. A fresh checkout reconstructs tracked source files. A smoke test is a small check that the main path starts and works. A limitation states a specific capability or guarantee the release does not provide.

Choose the branches you implemented and write a demonstration around their exact behavior. The release can be the core alone, a networked simulator, or a selected performance/concurrency extension. Deferred lessons remain optional and do not block a correct first release.

Make the fresh-build path concrete

Record the MSYS2 UCRT64 package versions, g++ banner and target, CMake/Ninja/GDB versions, Windows version, dependency pin, and configuration files. The README begins with the L01 PATH step and the L02 configure/build/CTest commands. Include a small replay fixture and its expected output. A new checkout must not rely on untracked headers, stale generated binaries, or your editor's single-file settings.

When distributing executables, document the matching UCRT64 runtime requirement or package the runtime DLLs permitted by their licenses. A build that starts only from your prepared terminal is still a valid development build, but it is not yet a self-contained binary distribution. Do not confuse that packaging issue with a matching bug.

A reproducible five-part demonstration

1. Start from an empty book and show the L05 three-trade sweep with exact maker IDs and prices.
2. Show a same-price size reduction preserving FIFO, then a size increase losing FIFO priority on a fresh fixture.
3. Replay the saved command file twice and compare every result and final snapshot.
4. Run a minimized regression test for one real defect and explain why it failed before the fix.
5. Show one baseline/candidate measurement with its workload, raw repetitions, units, and keep/revert decision.

If networking exists, append a fragmented-frame example and one dropped-feed-event recovery. If queues exist, show close/drain under output backpressure. If sharding exists, compare per-symbol histories and state whether global merge order is persisted. If persistence exists, demonstrate the exact completed clean or crash-recovery boundary.

Audit claims against evidence

Claim	Evidence to retain
Correct price-time matching	Literal traces plus independent reference comparisons
Indexed cancellation	Locator design, lifecycle tests, scoped complexity statement
Faster storage variant	Same input/state, raw repeated measurements, memory cost
Network framing	Every-byte splits, coalescing, disconnect and short-write tests
Recovery	Snapshot position and exact applied suffix under injected loss
Safe thread handoff	Ownership table, ordering argument, full/empty/shutdown tests

Use language such as 'validated against an independent reference on saved workloads' or 'measured X commands/s for workload Y on machine Z.' Do not replace a named measurement with 'ultra-low latency' or treat Checked as ASan/TSan evidence. Keep limitations specific: one symbol, bounded outputs, no authentication, or clean-restart-only persistence, according to the actual release.

Keep progress evidence usable

Mark a lesson Done when you can explain its mechanism and its checks pass. Record a test name, trace path, benchmark record, or commit in the existing fillable fields. The short exercises in these teaching pages have answers included; the code and exact project outputs remain your implementation work. Use the session log to record the next observable action, not an open-ended goal such as 'work on networking.'

Exercise: a pooled variant is slower but passes every correctness test, and you revert it with measured evidence. Answer: P01 can still be Done. You completed the experiment and learned why the baseline fits this workload. Tag the selected release once every claimed capability has evidence, and continue applications without waiting for every optional extension.

X03 Demonstrate the integrated Windows simulator

Prerequisite: completed branches selected for your release. Suggested work unit: one or two focused sessions.

Understand the mechanism

Integration is complete when another person can build the selected feature set, run a small demonstration, reproduce a failure-recovery scenario, and understand the measurement boundary. Preserve a simple in-process path so sockets and threads never become prerequisites for testing matching behavior.

Work through an example

A useful demonstration starts the server and subscriber, submits the worked sweep, changes queue priority through an amendment, injects one dropped feed event, and shows the subscriber recovering to a matching snapshot. The explanation names the exact sequence at which the books agree.

Build it in this order

1. Create a release checklist listing only the implemented branches. Record the Windows environment, build commands, dependency versions, configuration files, and executable arguments.
2. Run a fresh-build smoke test, core regression suite, selected queue tests, and network fault cases. Test disconnect halfway through a frame, a slow reader, a feed gap, a full output queue, and shutdown during traffic where those features exist.
3. Write one architecture page naming state owners and command/result boundaries. Add performance tables with raw-run links, baseline commits, and the interpretation of each rejected optimization.
4. Record a short demo script and a limitations section. Tag the release only after every claimed capability has a test or demonstration. Leave unfinished extension rows Deferred or Active.

Check the result

A fresh Windows checkout follows the documented commands and reaches the demo. Every stated performance number has a scope and raw evidence. You can explain ownership, replacement behavior, replay, and one measured design trade-off without relying on a generated implementation.

If it fails: If the demo requires undocumented manual setup, add it to the instructions. If only networking tests expose a core bug, minimize the command stream into an in-process regression. If a benchmark cannot be reproduced, remove the claim until its measurement is understood.

Use the preceding teaching pages, then record understanding and verification below.

☐ I can explain the mechanism. ☐ The completion checks pass.

Evidence or commit: Actual minutes:

Session log and next action

Choose one lesson and an observable result. After each session, record the outcome and next action. When this log fills, save a dated copy, then clear only these log fields for new entries.

Session 1

Date: Lesson: Minutes:

Intended result

Evidence or failing test

Blocker and next action

Session 2

Date: Lesson: Minutes:

Intended result

Evidence or failing test

Blocker and next action

Session 3

Date: Lesson: Minutes:

Intended result

Evidence or failing test

Blocker and next action

Adjust the plan from evidence

If a lesson spans several sessions, split its remaining steps. If the build is broken, the next action is a minimal reproducer or toolchain check. If an experiment shows no useful improvement, recording and rejecting it completes that experiment. Progress counts understanding and evidence, not only added features.

Bug and experiment records

Bug record

Lesson and date

Expected rule

Observed behavior

Smallest input and initial state

Root cause

Fix and regression test

Commit

Experiment record

Hypothesis and single change

Baseline commit and candidate commit

Workload file and initial state

Compiler build CPU and power mode

Repeated results and units

Keep or revert and reason

Store full traces and benchmark JSON in the repository and refer to their paths here. Do not paste megabytes of output into the workbook. The document is your progress record; the repository stores executable evidence.

Sources and optional reference

All required teaching is in the lessons. Links provide attribution and optional detail. Platform references were checked on 14 September 2026; record installed tool versions. Simulator policies, worked examples, and the custom wire formats are workbook choices.

[1] MSYS2 updates and tool packages

Install/update the UCRT64 tool family; repeat a full update after a requested terminal restart.

[2] CMake compile features and MSYS2 CMake

Use native UCRT64 CMake with Ninja; language requirements belong to the CMake project.

[3] VS Code: GCC with MinGW

Compiler paths, UCRT64 installation, and GDB launch configuration.

[4] GoogleTest CMake quickstart

Library/test targets and discovery. The workbook provides its complete scaffold.

[5] libstdc++ debug mode

Checked-container behavior and compatibility of container representations across compilation units.

[6] GCC instrumentation options

Instrumentation is target/runtime dependent. This workbook requires no sanitizer runtime.

[7] C++ fundamental integer types

Unsigned arithmetic and the need for explicit range checks.

[8] C++ condition variables

Predicate waits, locking, and notifications for C01.

[9] Investor.gov: order types

Basic market/limit terminology. Matching and amendment policies here are simulator choices.

[10] C++ container requirements

List, deque, ordered-map and unordered-map lifetime and complexity rules.

Sources and optional reference continued

[11] Google Benchmark user guide

Timing scope, repetitions, setup, and optimization barriers.

[12] Windows high-resolution timing

Interval measurement, timer cost, and clock limitations.

[13] C++ pool memory resources

Pool reuse, upstream storage, and resource lifetime.

[14] GNU gprof manual

Optional g++ profiling experiment. Confirm support with a smoke test first.

[15] WPR command-line options

CPU recording, file mode, and saving a trace.

[16] Windows Performance Analyzer

CPU/scheduling analysis. GCC symbol resolution must be verified for the installed tools.

[17] Nasdaq OUCH order-entry reference

Optional order-entry comparison. E01/E03 define a separate custom protocol.

[18] BX TotalView-ITCH 5.0, 5 December 2014

X01 field offsets and visible-order lifecycle semantics; synthetic fixture subset is included.

[19] Winsock WSStartup

Initialization, negotiated version, direct startup error, and cleanup lifetime.

[20] Winsock recv

Positive byte counts, EOF, and socket errors.

Sources and optional reference continued

[21] Winsock send

Partial sends and the distinction between local acceptance and peer processing.

[22] Winsock WSAPoll

Readiness flags and bounded nonblocking service loops.

[23] Winsock recvfrom

Datagram boundaries, buffer limits, and errors.

[24] Nasdaq MoldUDP64 reference

Sequencing/batching concepts. N04 specifies its own envelope.

[25] Nasdaq GLIMPSE reference

Snapshot/stream handoff concept. N05 defines its own records.

[26] Windows IP multicast

Membership, interface selection, and Windows receive-side multicast loopback behavior.

[27] C++ data races and synchronization

Ownership, conflicting accesses, and happens-before.

[28] C++ atomic ordering

Sequential consistency and acquire/release publication.

[29] Windows FlushFileBuffers

The file-flush operation and its limits; crash durability needs a tested boundary.

[30] libstdc++ configuration macros

Debug-container and assertion definitions; consistent settings across the Checked build.